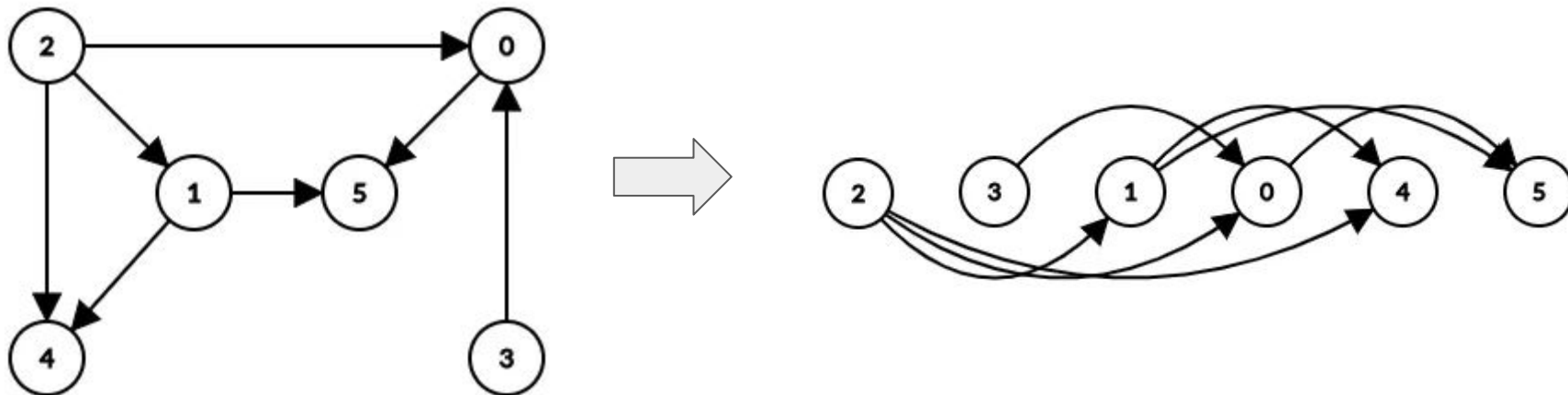


Grafos III

Ordenamiento topológico, componentes fuertemente
conexas y el algoritmo de Tarjan

Ordenamiento Topológico

Es una ordenación lineal de los vértices de un grafo dirigido acíclico (DAG) tal que para cada arista $u \rightarrow v$, el vértice u aparece antes que v en el orden topológico.



Ordenamiento Topológico (DFS)

```
void dfs(int node) {  
    for (int next : graph[node]) {  
        if (!visited[next]) {  
            visited[next] = true;  
            dfs(next);  
        }  
    }  
    top_sort.push_back(node);  
}
```

En el main:

```
visited = vector<bool>(n);  
for (int i = 0; i < n; i++) {  
    if (!visited[i]) {  
        visited[i] = true;  
        dfs(i);  
    }  
}  
std::reverse(top_sort.begin(), top_sort.end());
```

Ordenamiento Topológico (DFS)

Para revisar si el orden topológico es válido:

```
vector<int> ind(n);  
for (int i = 0; i < n; i++) { ind[top_sort[i]] = i; }  
  
// Check if the topological sort is valid  
bool valid = true;  
for (int i = 0; i < n; i++) {  
    for (int j : graph[i]) {  
        if (ind[j] <= ind[i]) {  
            valid = false;  
        }  
    }  
}  
}
```

Un grafo NO tendrá un orden topológico válido cuando tiene algún ciclo.

Código: [USACO Guide](#)

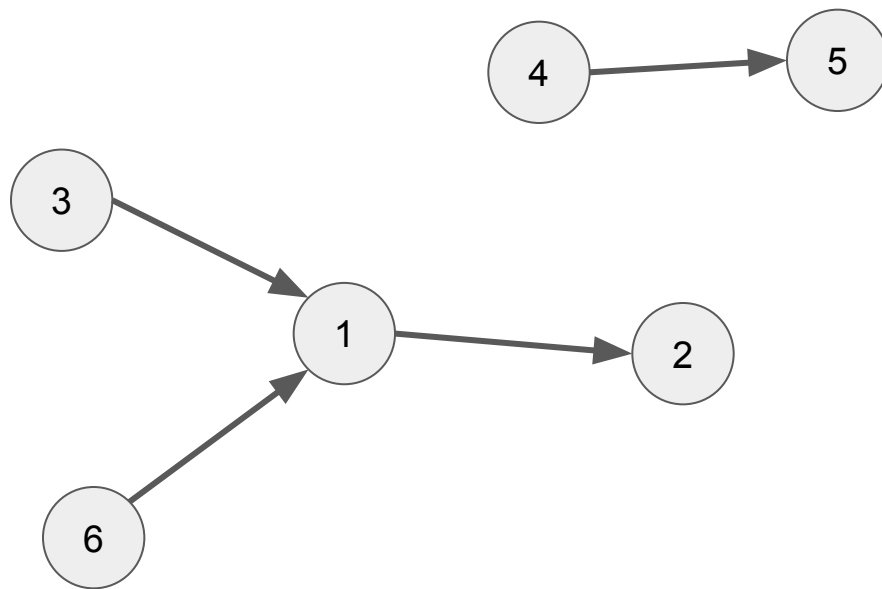
Ordenamiento Topológico (BFS / Algoritmo de Kahn)

El algoritmo consiste en siempre visitar los nodos cuyo *indegree* (o grado positivo) sea 0. Una vez los visitamos, los insertamos en el orden topológico, “quitamos” los nodos y continuamos el algoritmo hasta visitar todos los nodos. Si el algoritmo tiene algún ciclo, no será posible visitar todos los nodos cumpliendo la restricción mencionada.

Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: { }

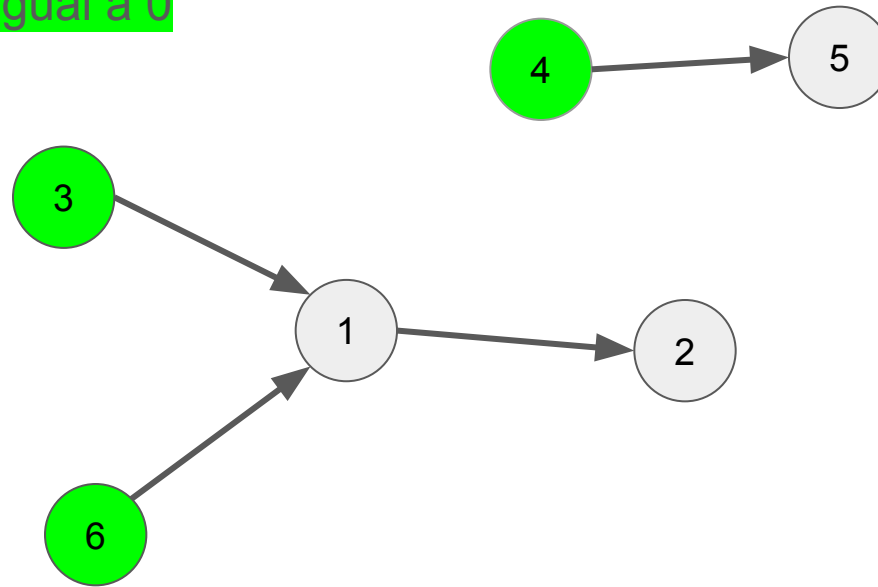


Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6}

Visitamos los nodos
con indegree igual a 0

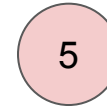


Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6}

Quitamos los nodos visitados. Notar que al eliminar nodos, tenemos que restar el indegree a los nodos vecinos de los que eliminamos.



Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6, 1, 5}

Visitamos los nodos con
indegree igual a 0.

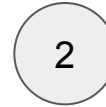


Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6, 1, 5}

Los quitamos.

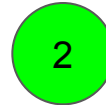


Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6, 1, 5, 2}

Visitamos los nodos con
indegree igual a 0.



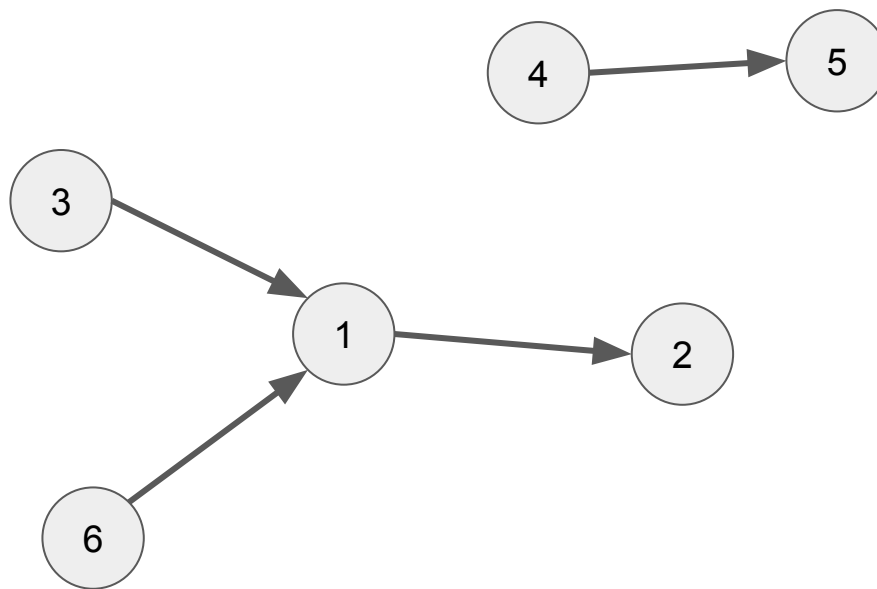
Ordenamiento Topológico (BFS / Algoritmo de Kahn)

Ejemplo:

Topo: {3, 4, 6, 1, 5, 2}

Listo, ya visitamos todos los
nodos.

Topo: {3, 4, 6, 1, 5, 2}



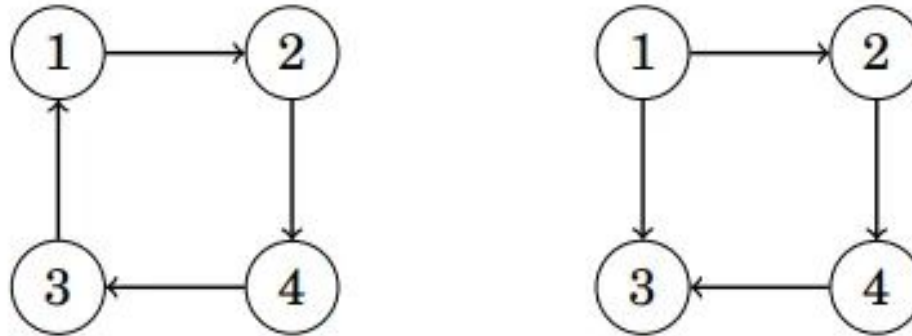
Propiedad del orden topológico

Una propiedad muy útil del orden topológico, es que podemos hacer programación dinámica (DP) sobre este orden topológico.

Problema de ejemplo: [Longest Flight Route](#)

Grafo fuertemente conexo

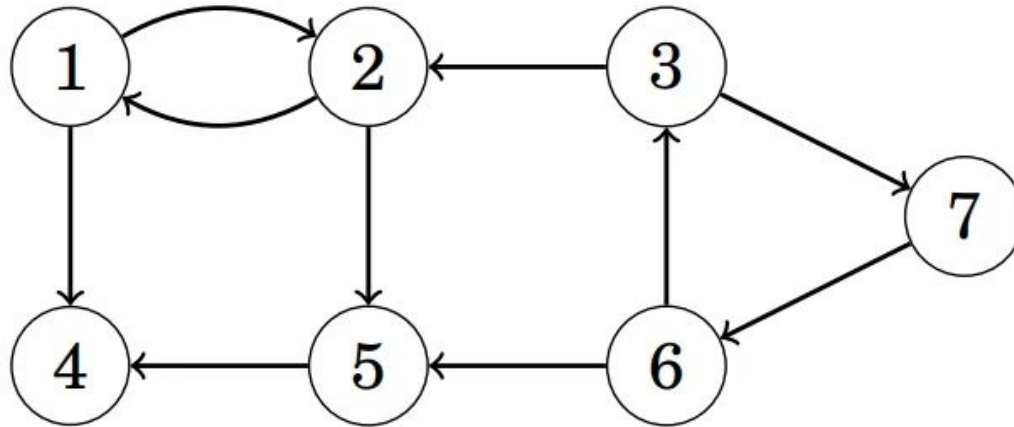
Un grafo se dice que es fuertemente conexo si existe un camino entre cualquier par de nodos del grafo (en ambas direcciones).



Por ejemplo en la imagen, **el de la izquierda sí es un grafo fuertemente conexo**, mientras que el de la derecha no lo es. (ya que del nodo 3 no podemos ir a ningún otro nodo)

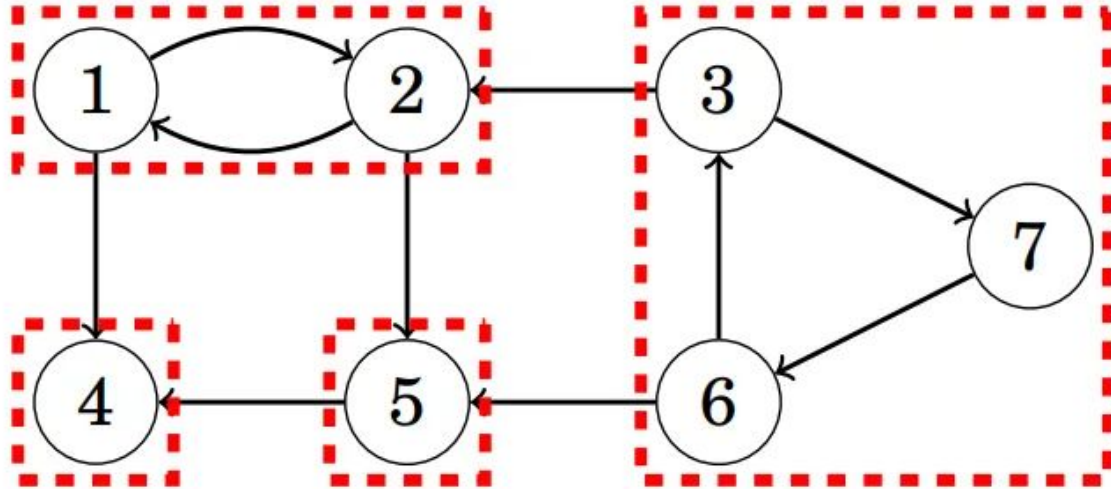
Componentes fuertemente conexas

Las **componentes fuertemente conexas de un grafo**, dividen un grafo en componentes fuertemente conexas que son lo más grandes posible. Cada componente fuertemente conexa forma parte de un DAG donde cada componente conexas es un nodo.



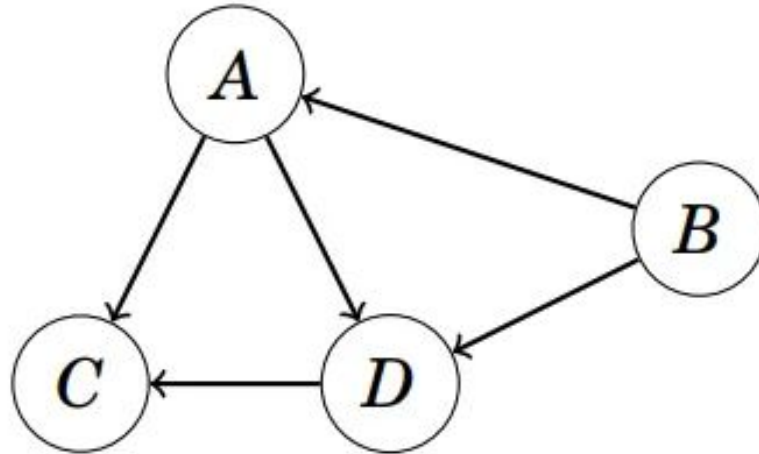
Componentes fuertemente conexos

Podemos ver cada componente conexa:



Componentes fuertemente conexos

Incluso podemos “condensar” el grafo, convirtiendo cada componente conexa en un nodo. Este grafo condensado siempre será un DAG.



Componentes fuertemente conexas

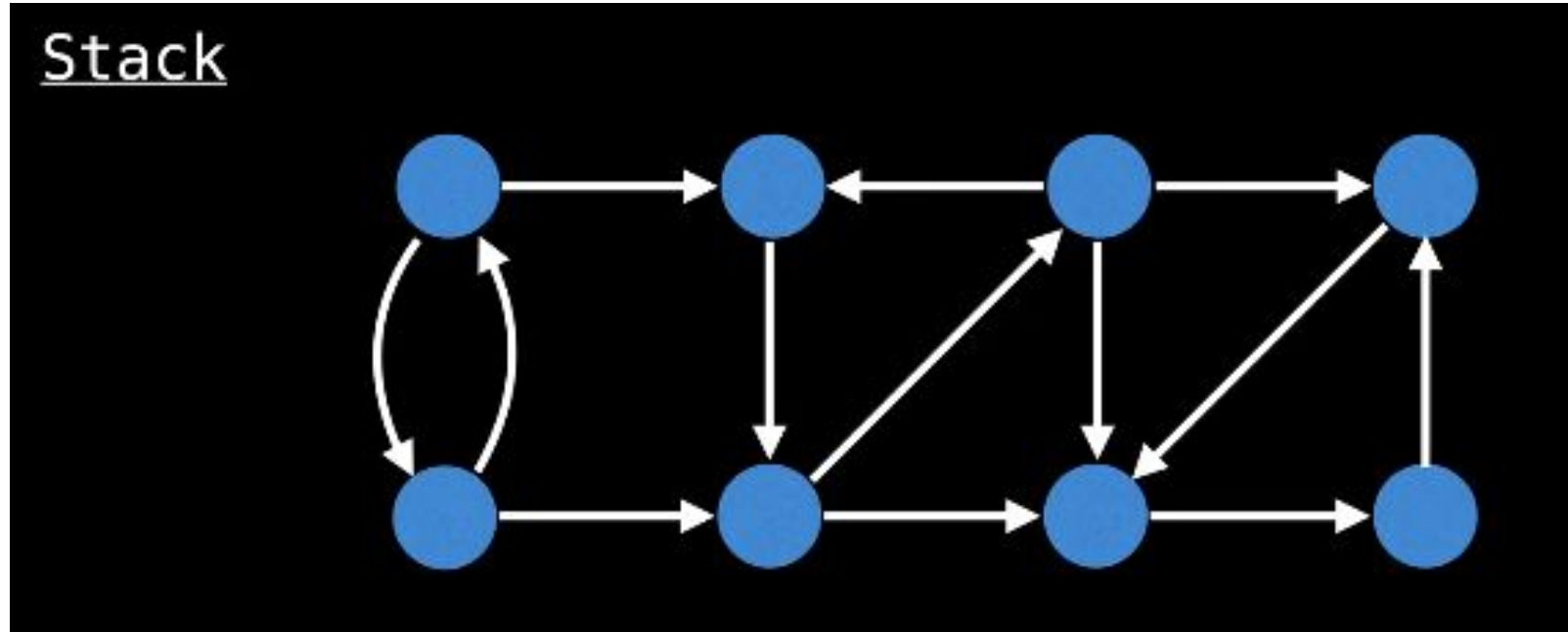
Existen dos algoritmos para resolver este problema: el **algoritmo de Tarjan** y el **algoritmo de Kosaraju**. Ambos algoritmos tienen una complejidad lineal $O(V + E)$, sin embargo, el de Tarjan usa solo un dfs, por lo que puede ser más cómodo de implementar, y algo más rápido que el de Kosaraju.

En esta oportunidad, vamos a ver el algoritmo de Tarjan.

Algoritmo de Tarjan

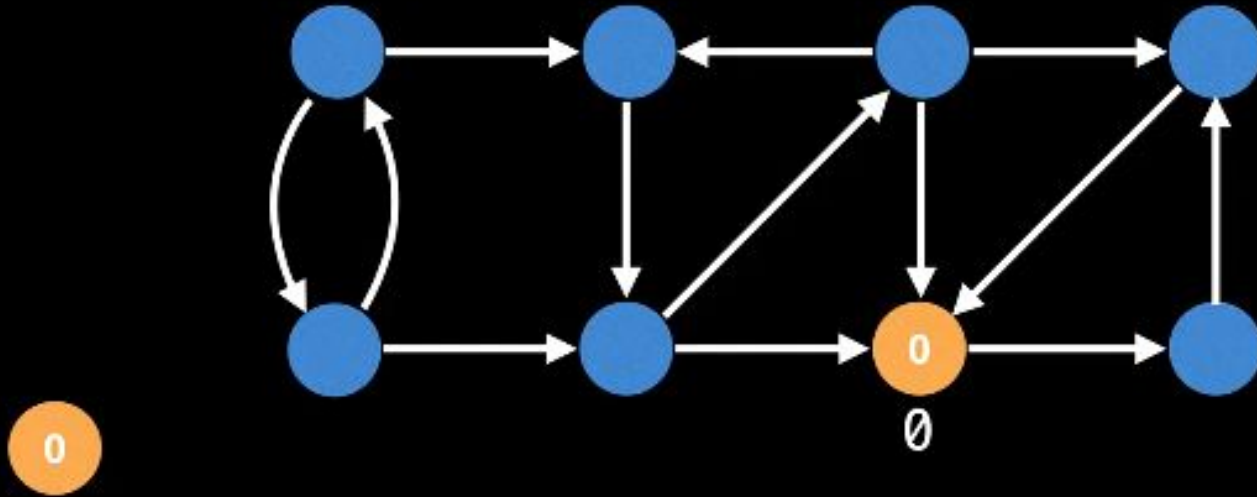
1. Marcamos todos los nodos como no visitados.
2. Ejecutamos un DFS. Al ejecutar un nodo, le asignamos valor a su id y a su low, ambos al mismo valor inicialmente.
3. Cuando un nodo termina de hacer dfs a un nodo vecino, actualizamos el valor de low del nodo actual. Digamos que el nodo actual es **u** y el nodo vecino es **v**:
 1. $\text{low}[u] = \min(\text{low}[u], \text{low}[v]);$
4. Luego de haber visitado a todos los vecinos de **u** (nodo actual), vamos a revisar si es que **u** es raíz o nodo inicial de una SCC. La condición para que esto sea verdad es si $\text{low}[u] == \text{id}[u]$.
5. Si es que este nodo es raíz de una SCC, entonces hemos encontrado una SCC completa. Vamos a quitar a todos los nodos de esta SCC del stack. Y podemos reportar que cada uno de estos nodos son parte de una misma SCC.

Algoritmo de Tarjan



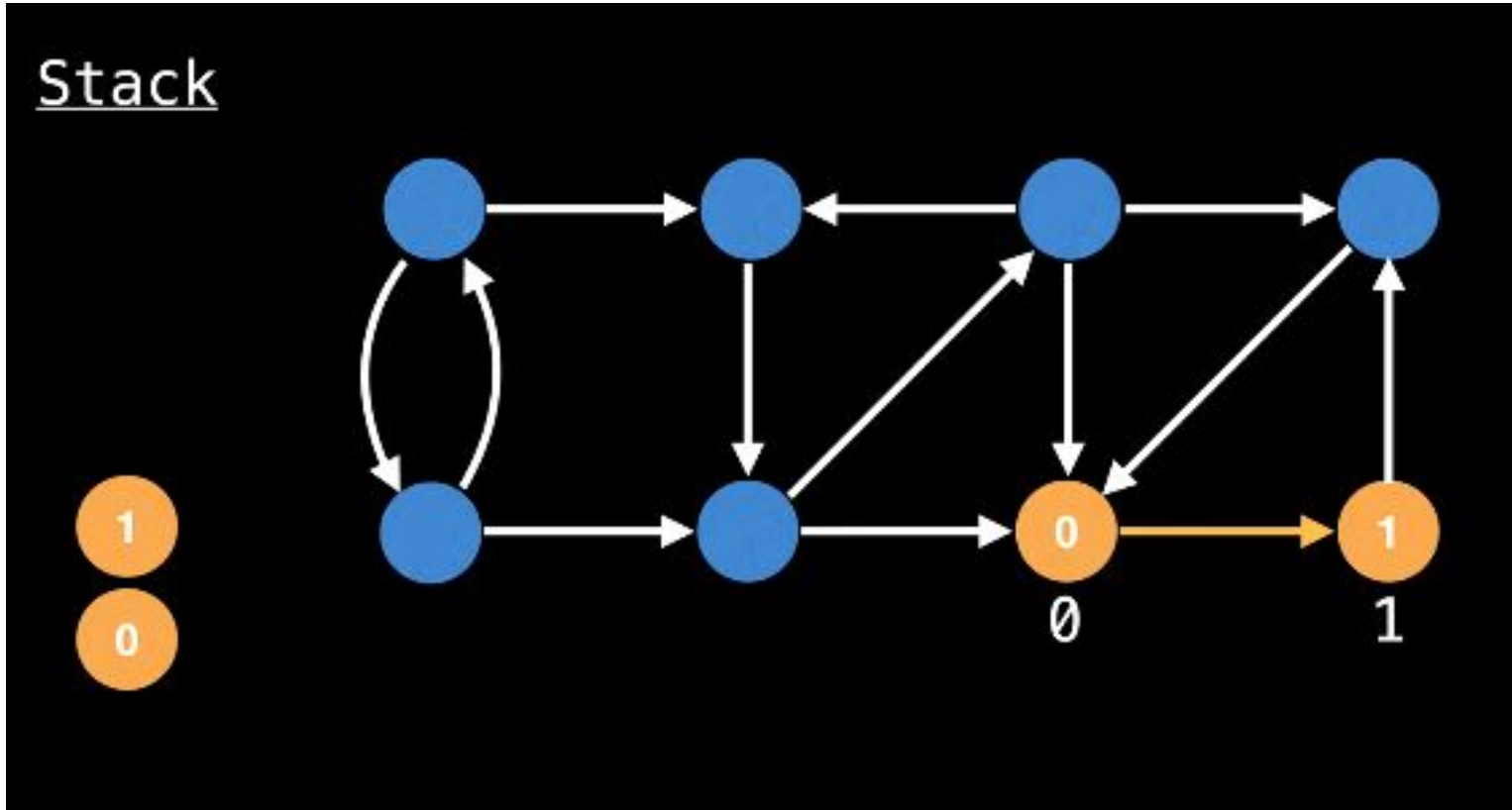
Algoritmo de Tarjan

Stack

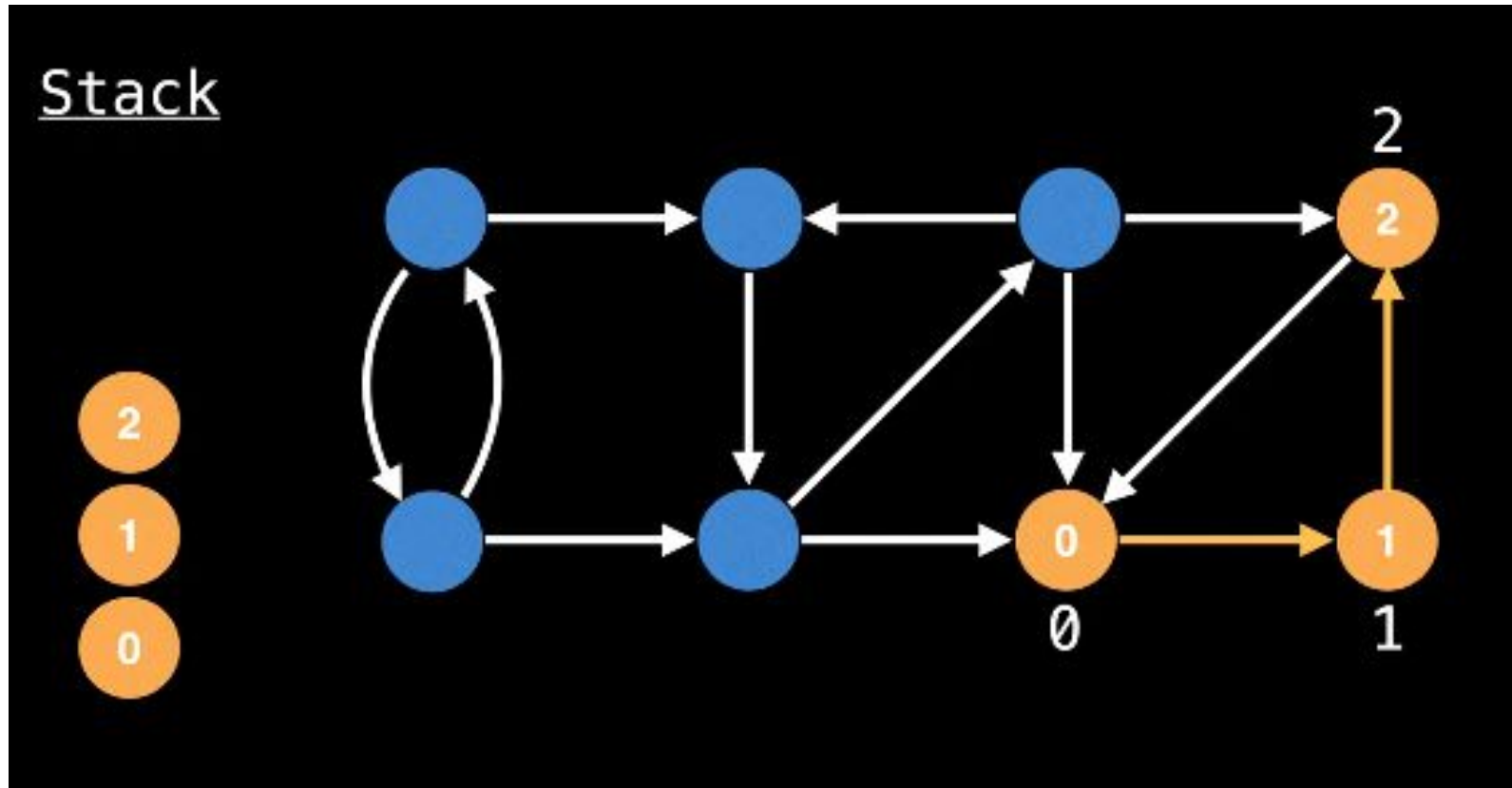


Start DFS anywhere.

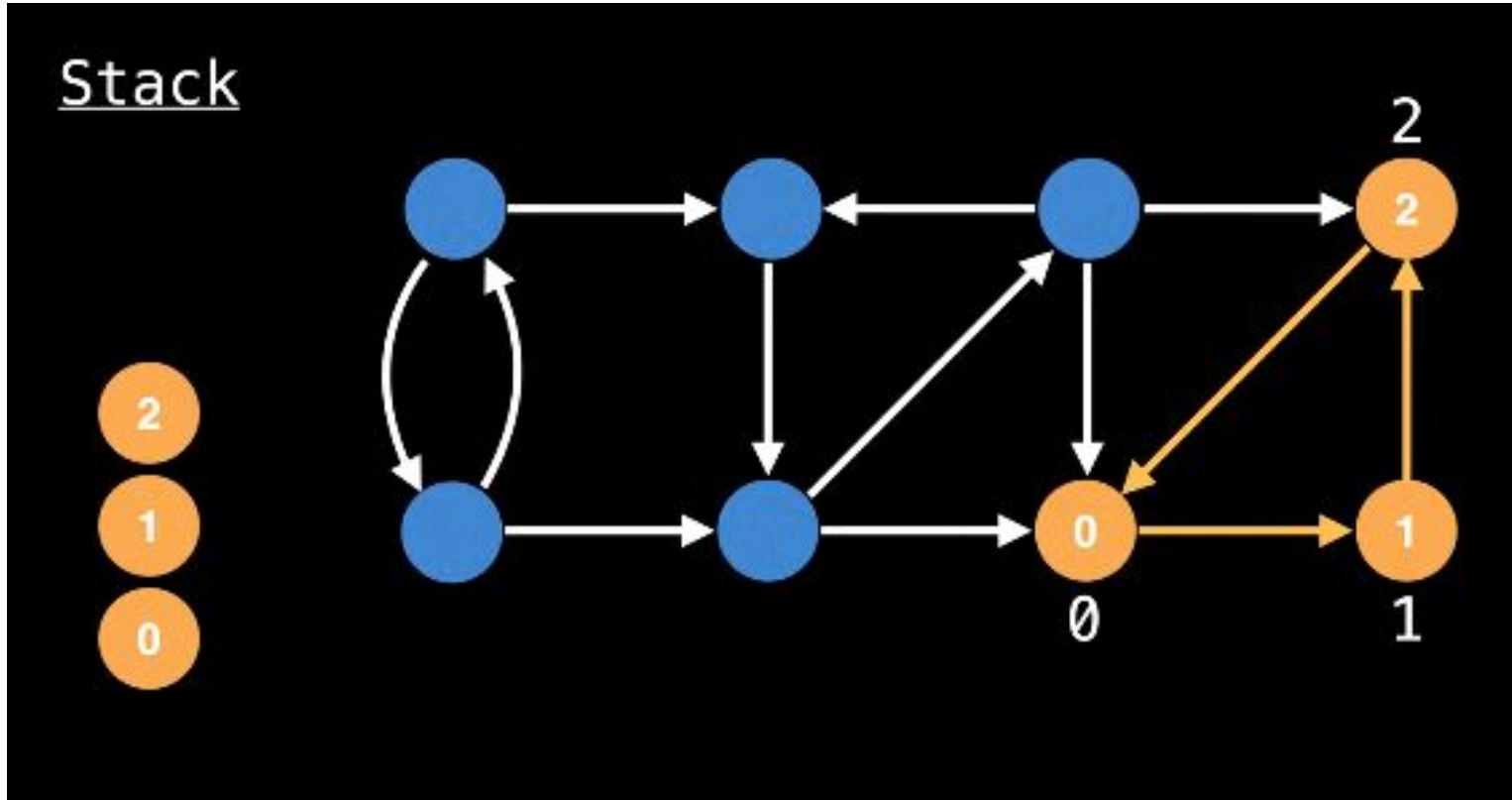
Algoritmo de Tarjan



Algoritmo de Tarjan

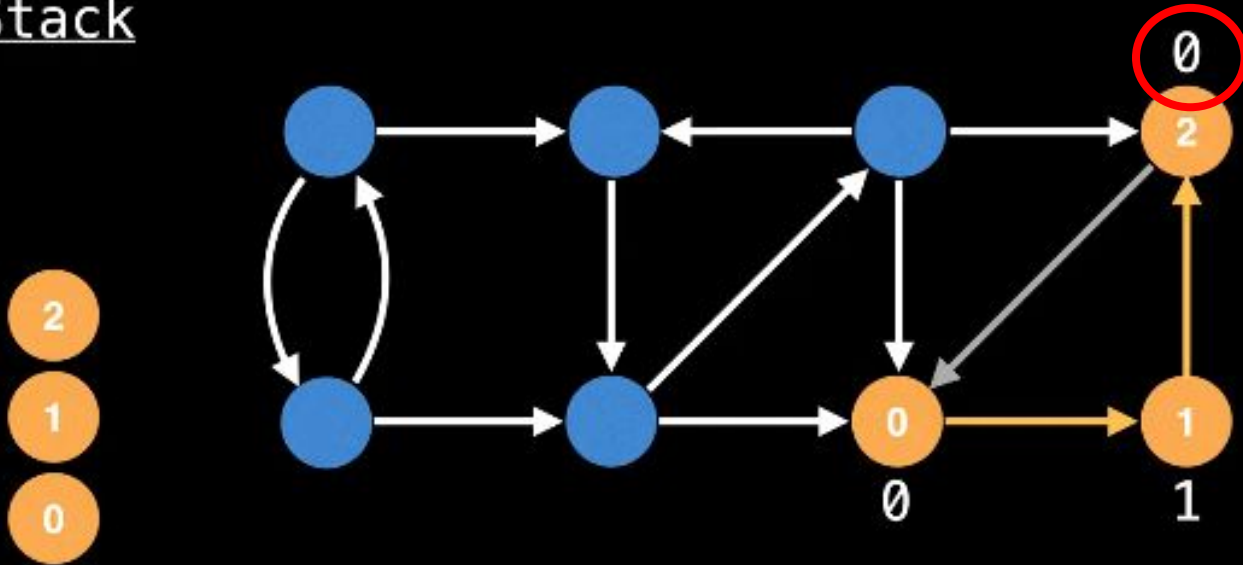


Algoritmo de Tarjan



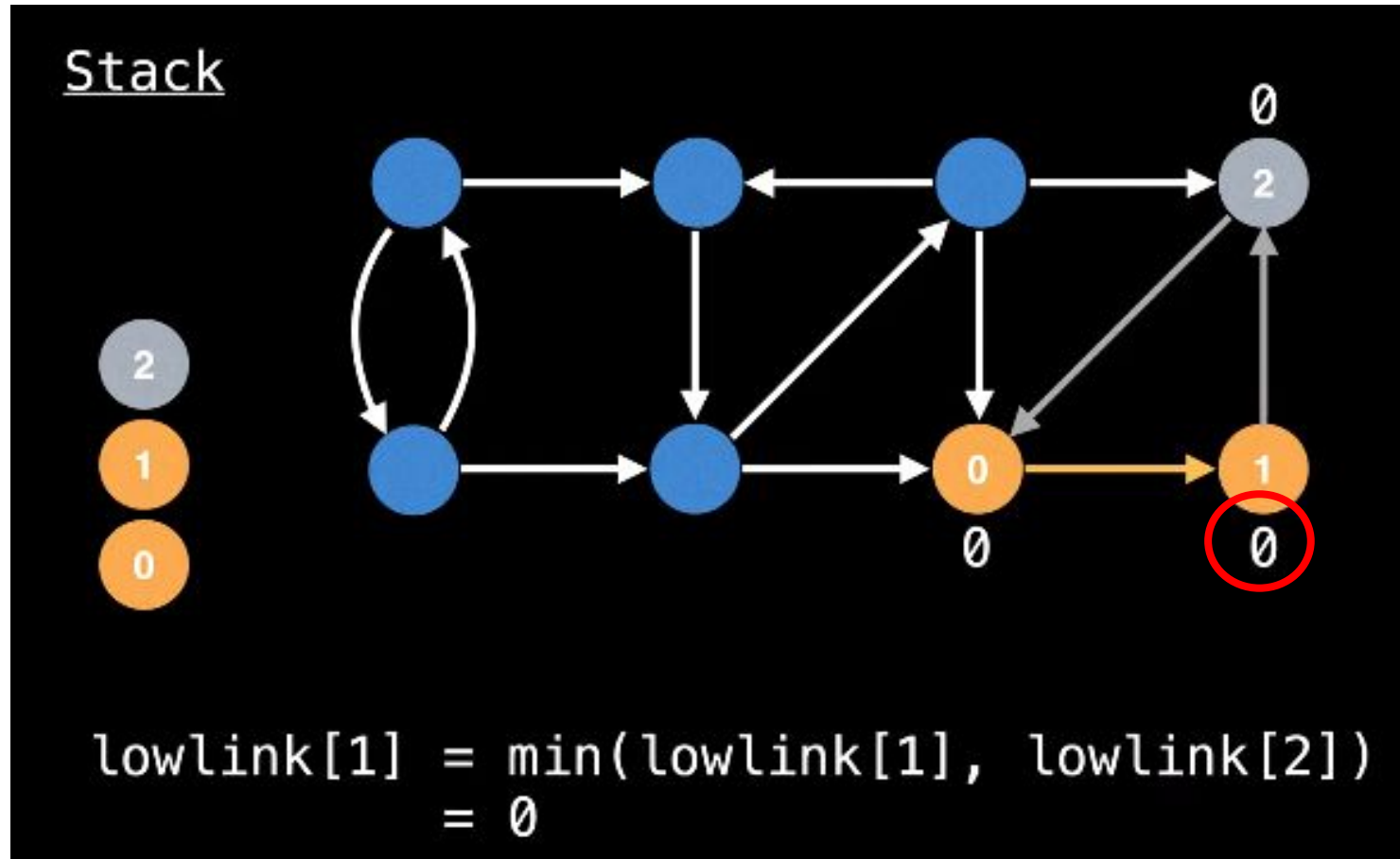
Algoritmo de Tarjan

Stack



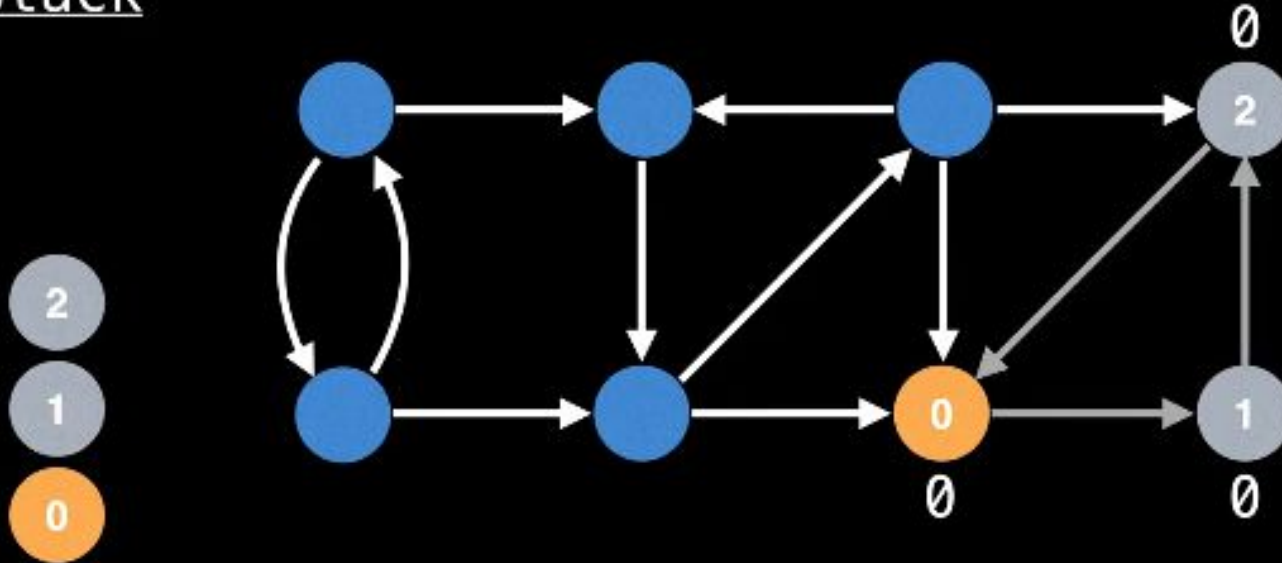
```
lowlink[2] = min(lowlink[2], lowlink[0])  
             = 0
```

Algoritmo de Tarjan



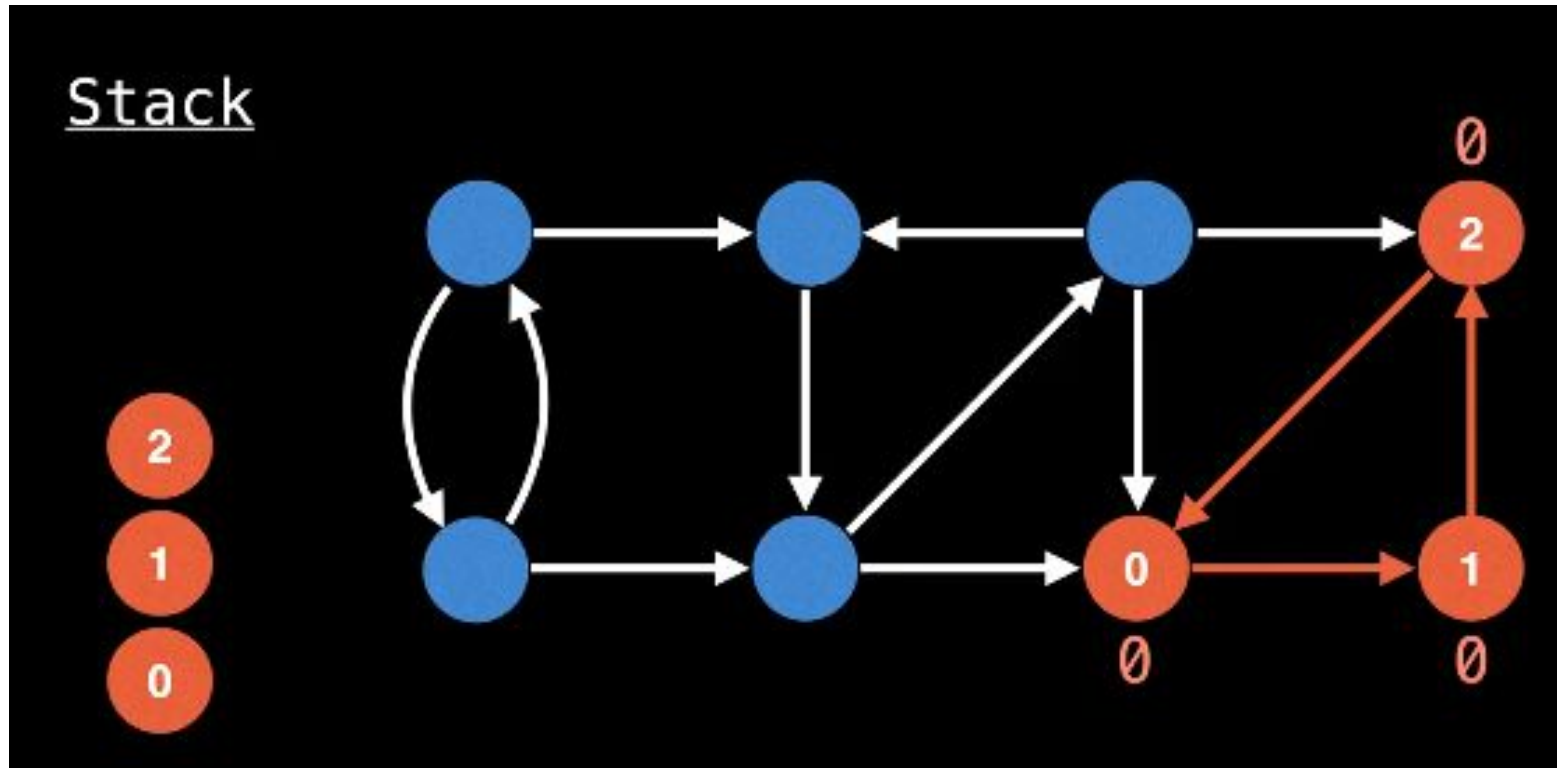
Algoritmo de Tarjan

Stack

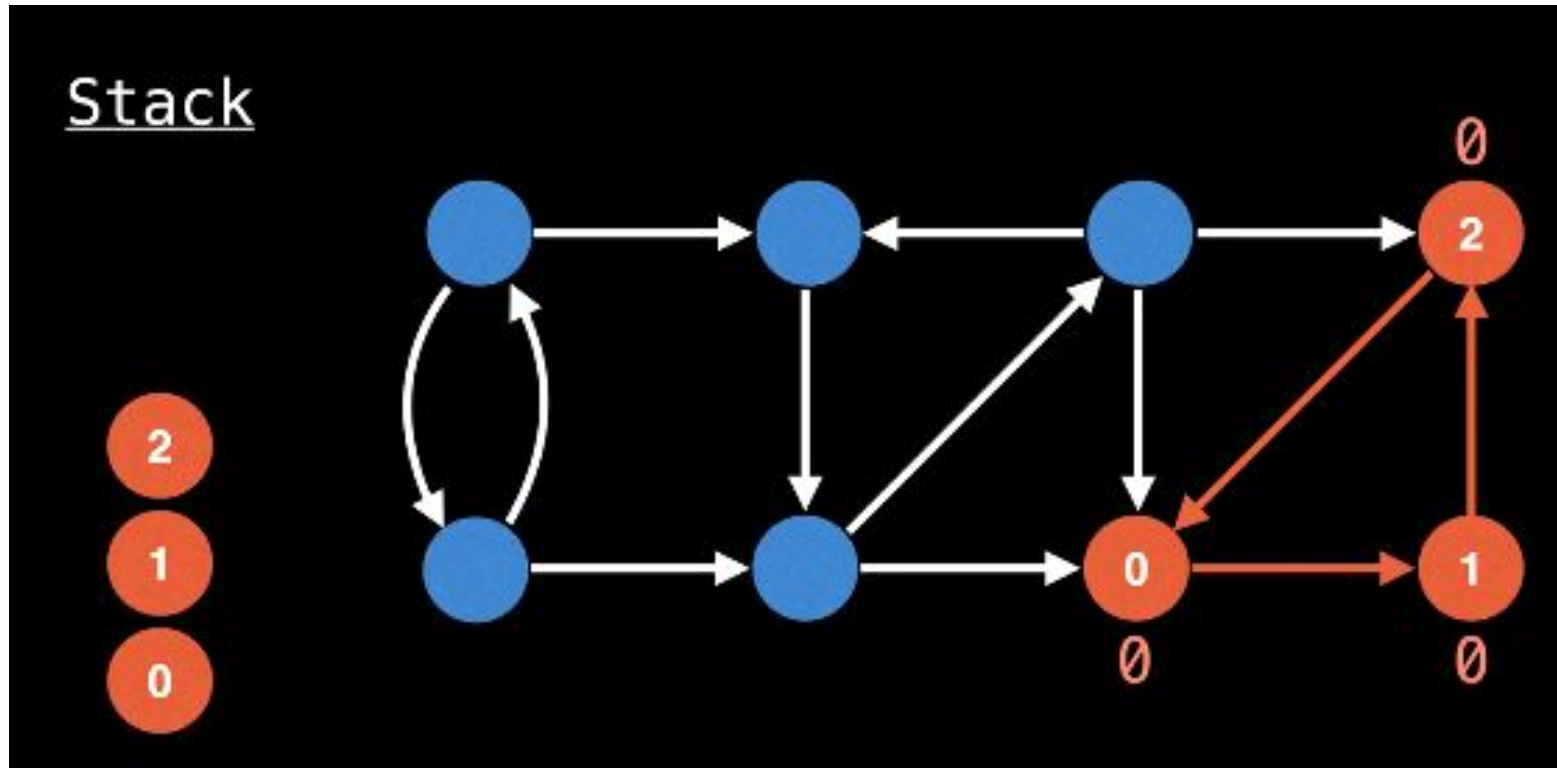


```
lowlink[0] = min(lowlink[0], lowlink[1])  
             = 0
```

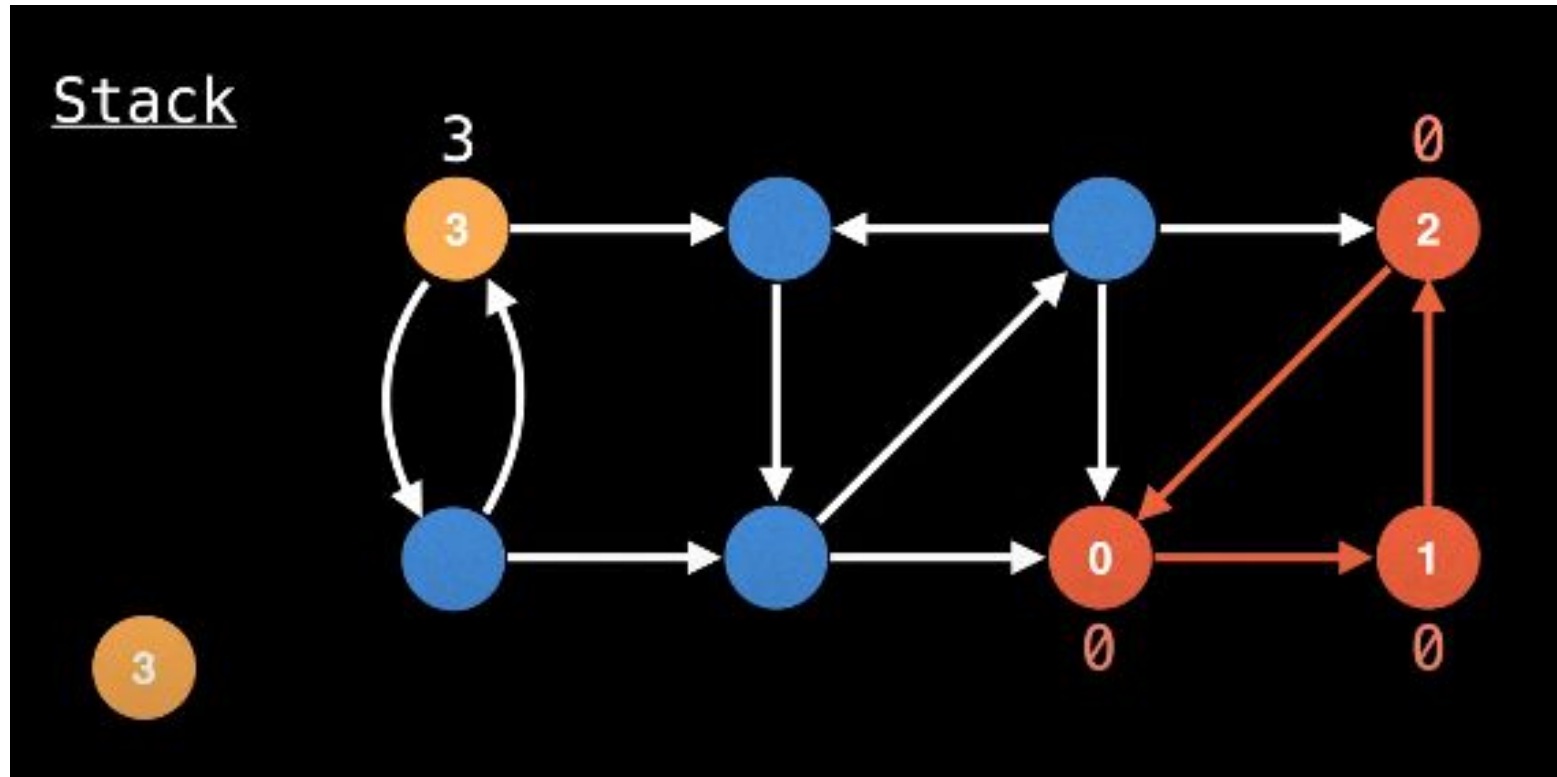
Algoritmo de Tarjan



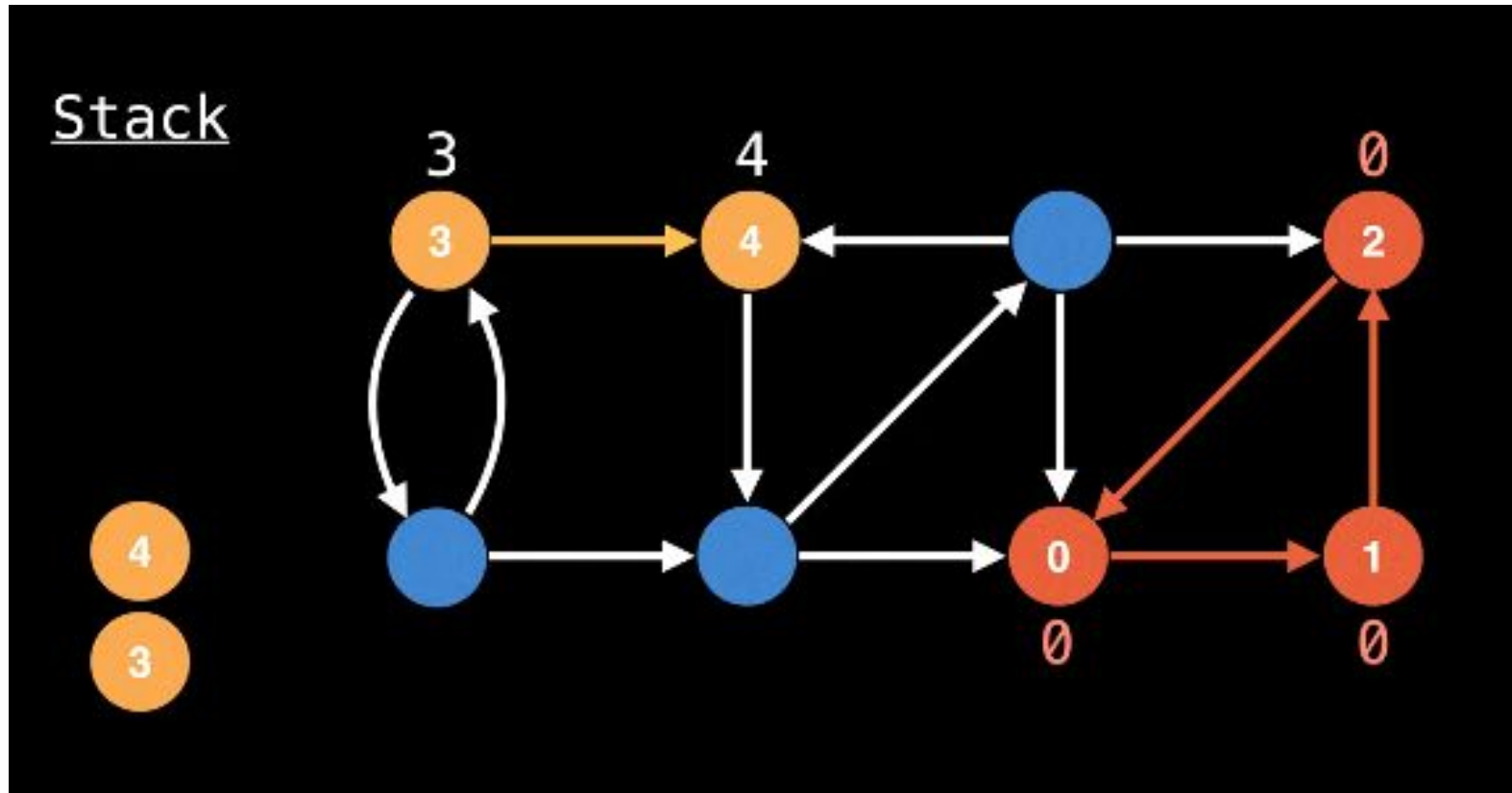
Algoritmo de Tarjan



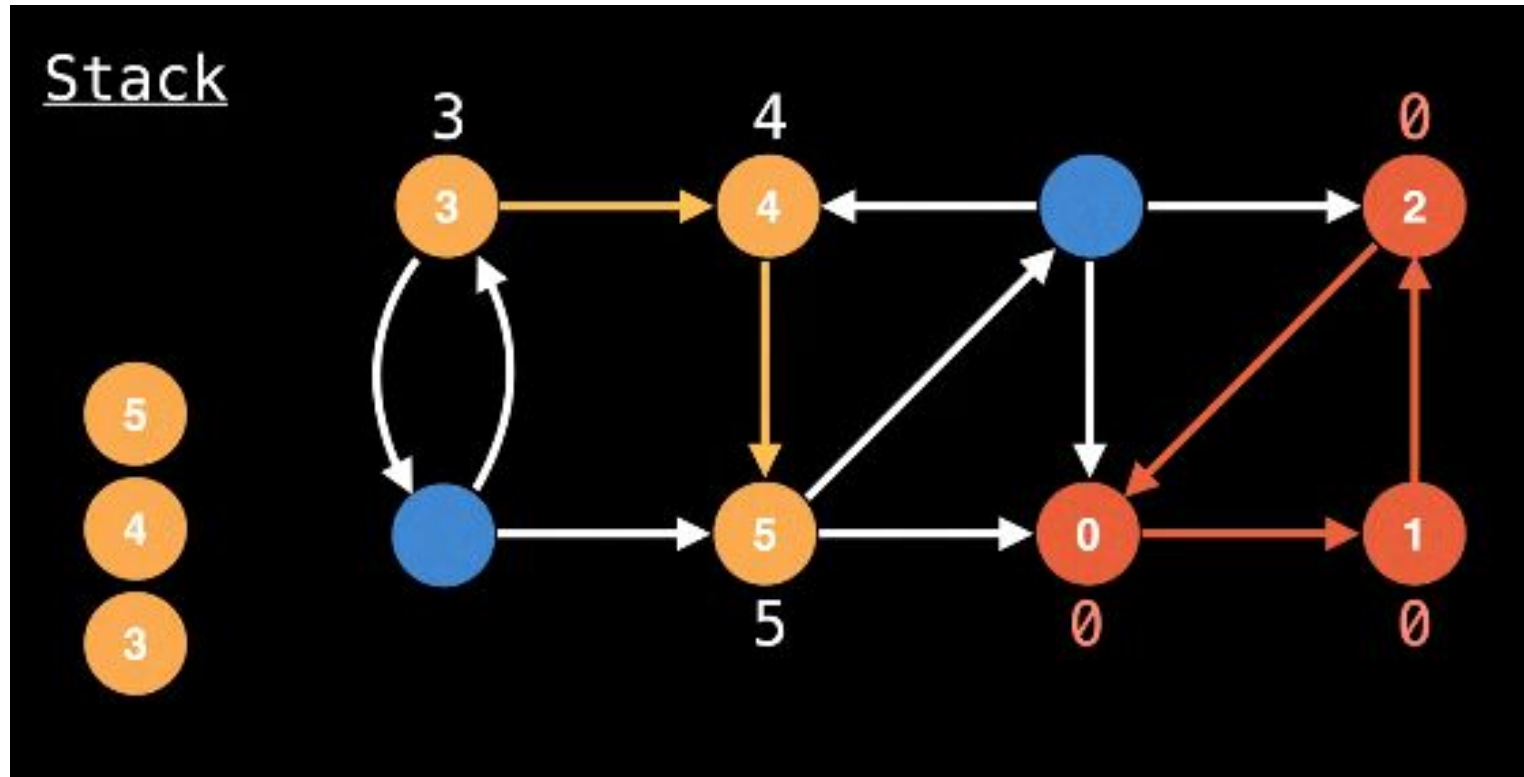
Algoritmo de Tarjan



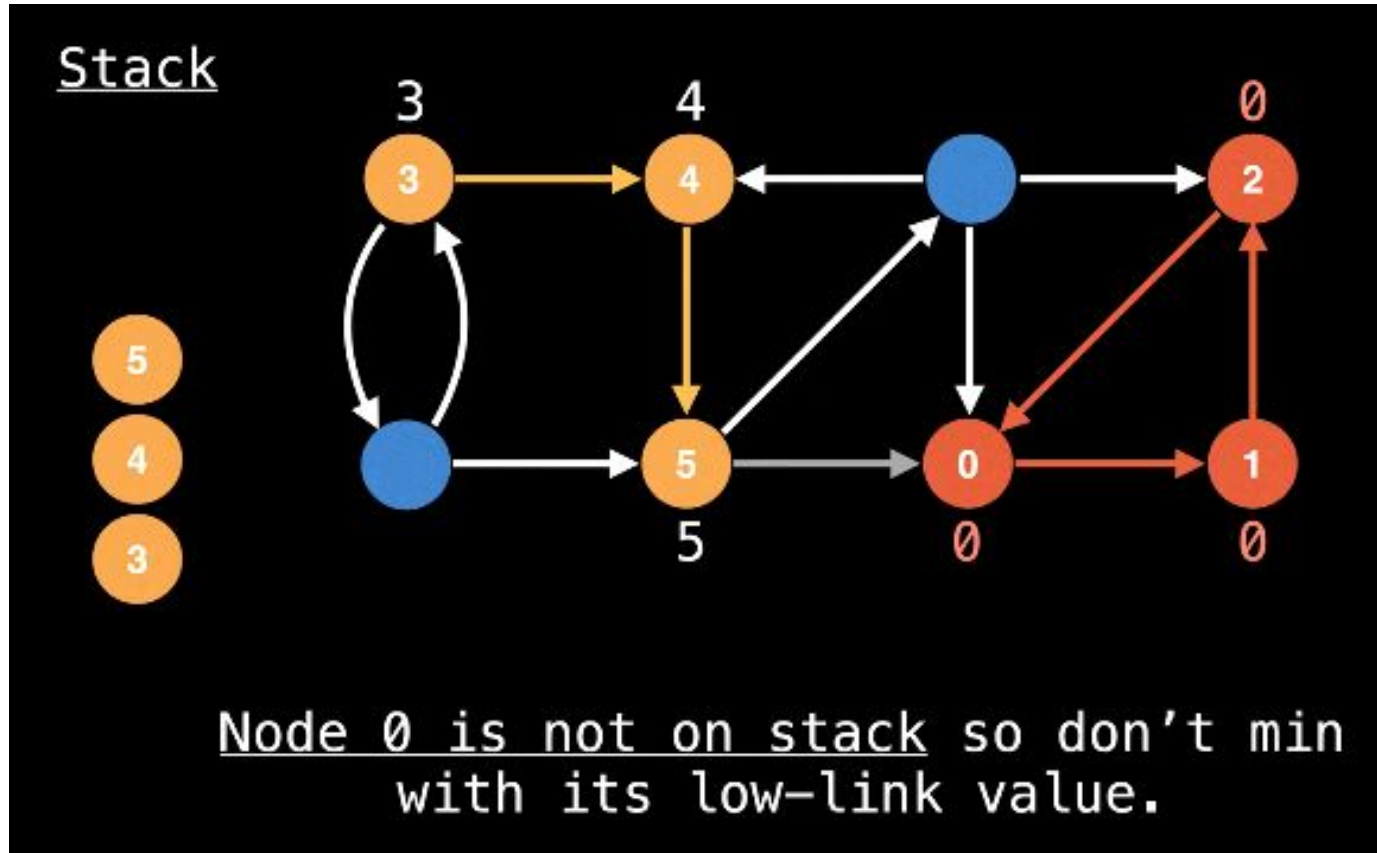
Algoritmo de Tarjan



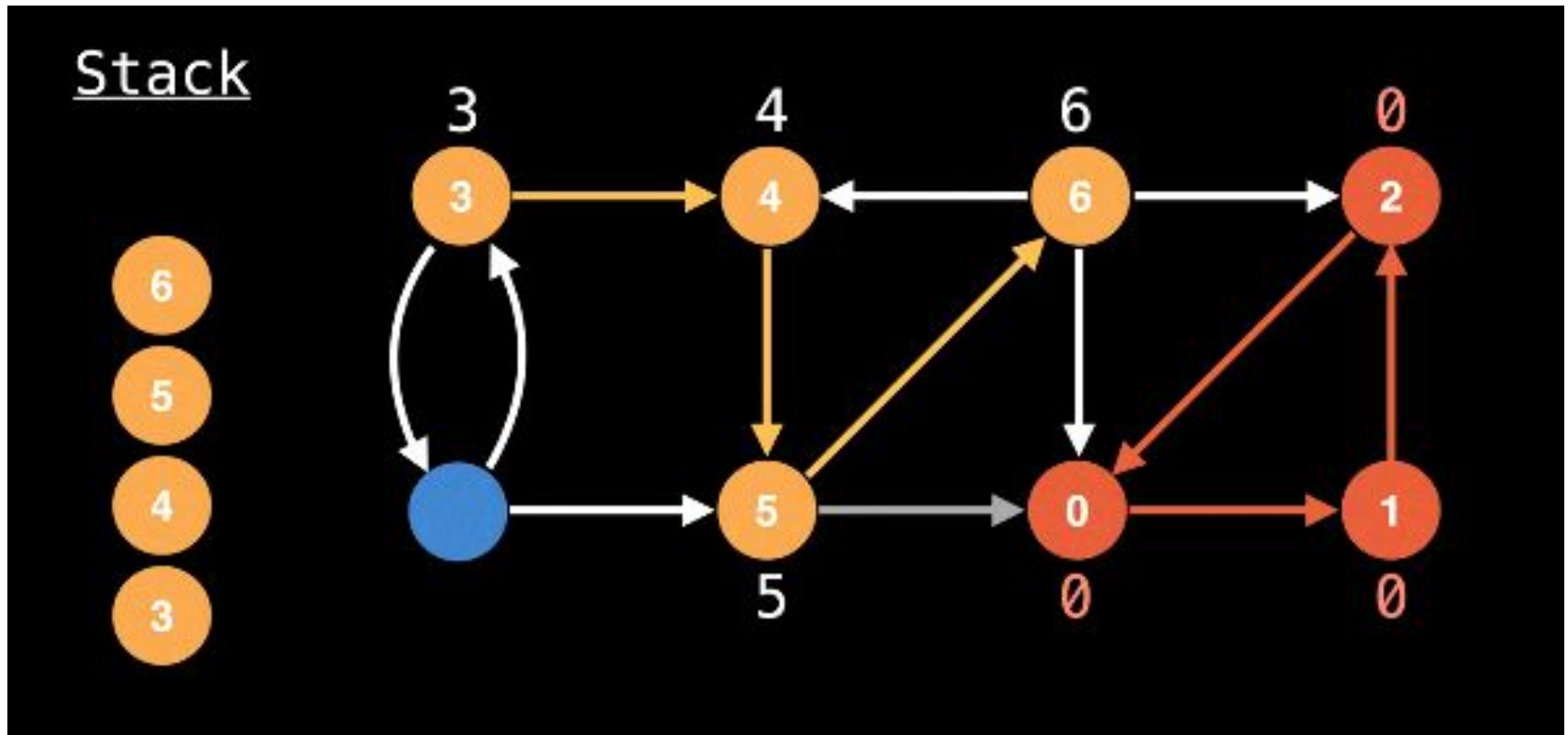
Algoritmo de Tarjan



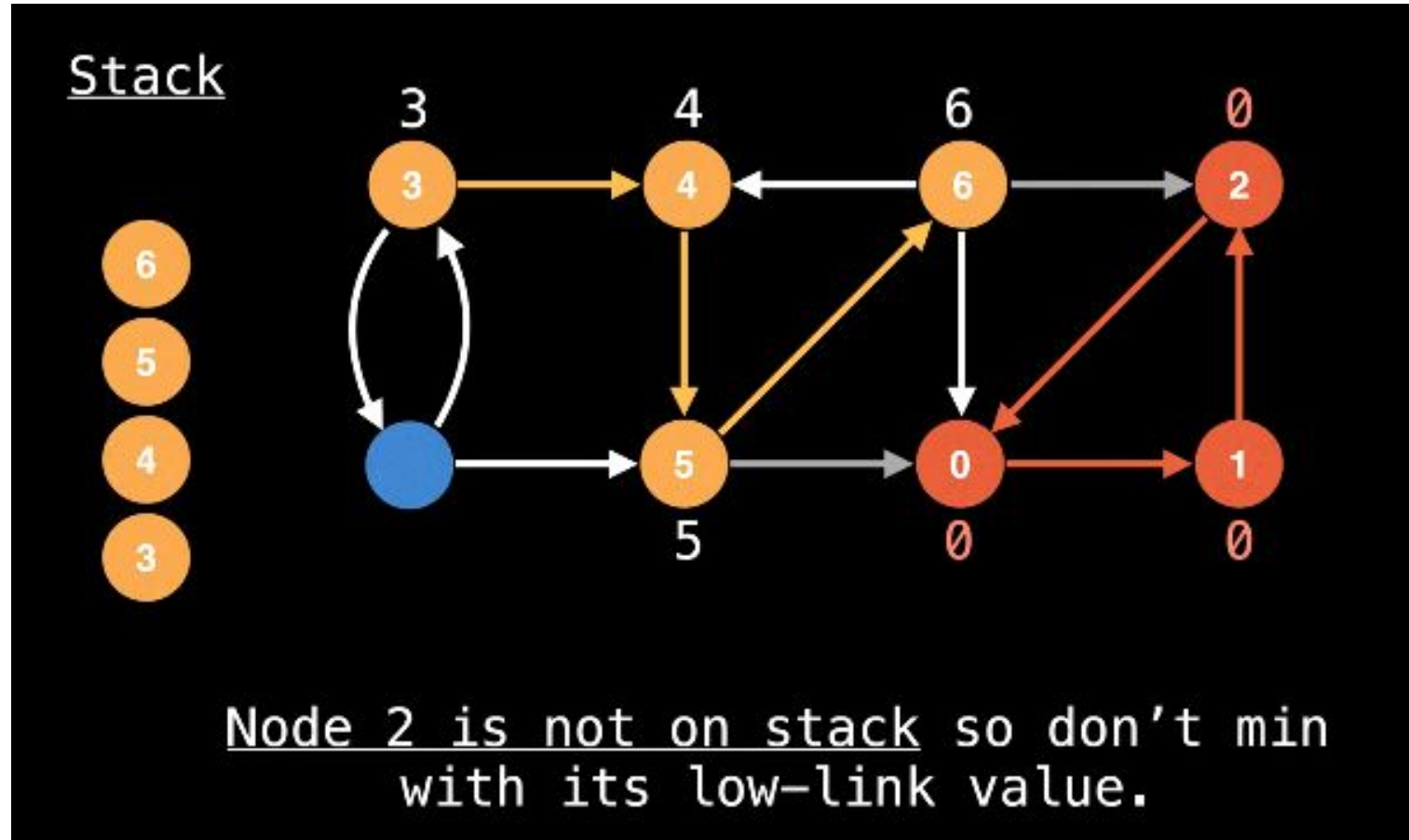
Algoritmo de Tarjan



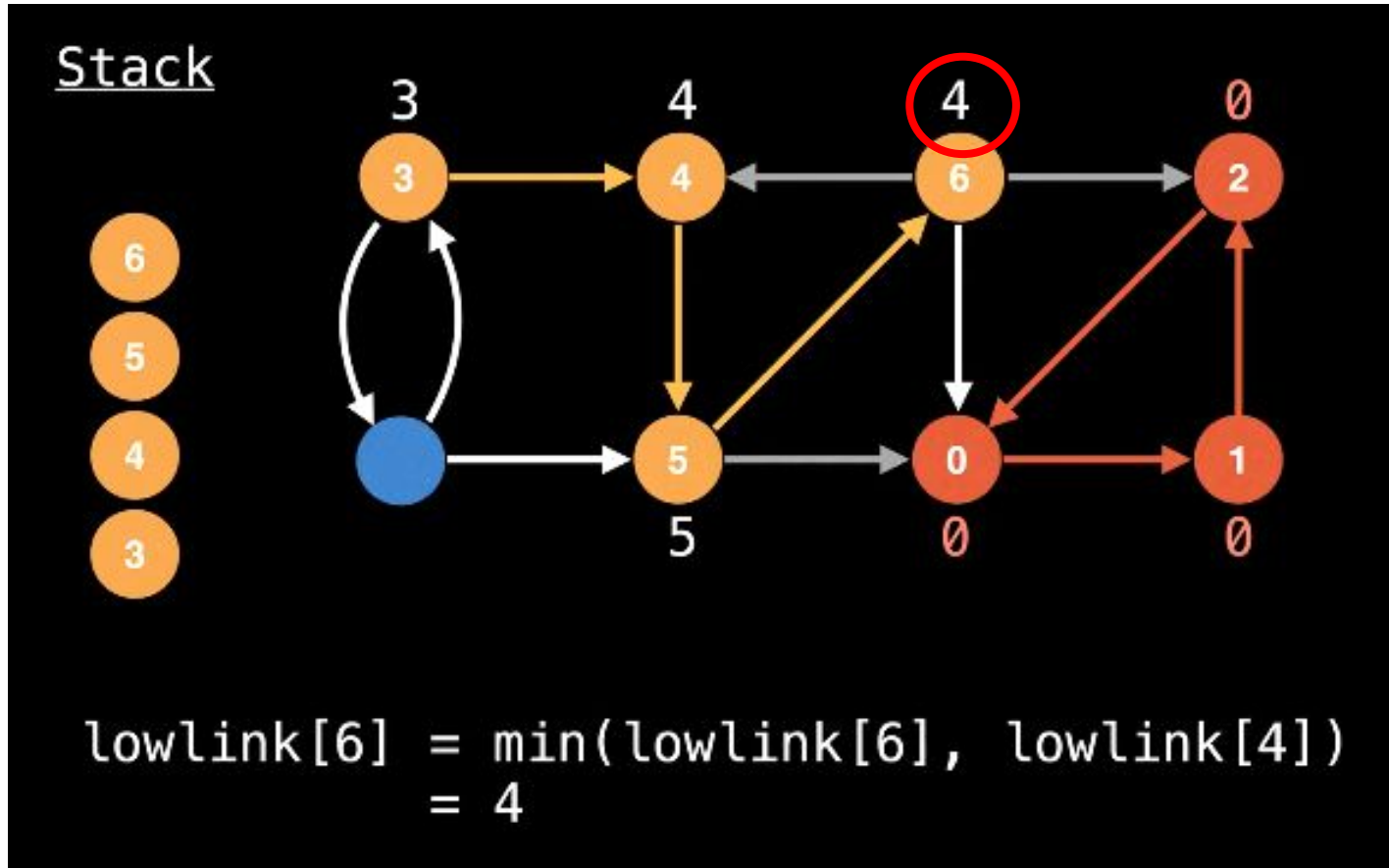
Algoritmo de Tarjan



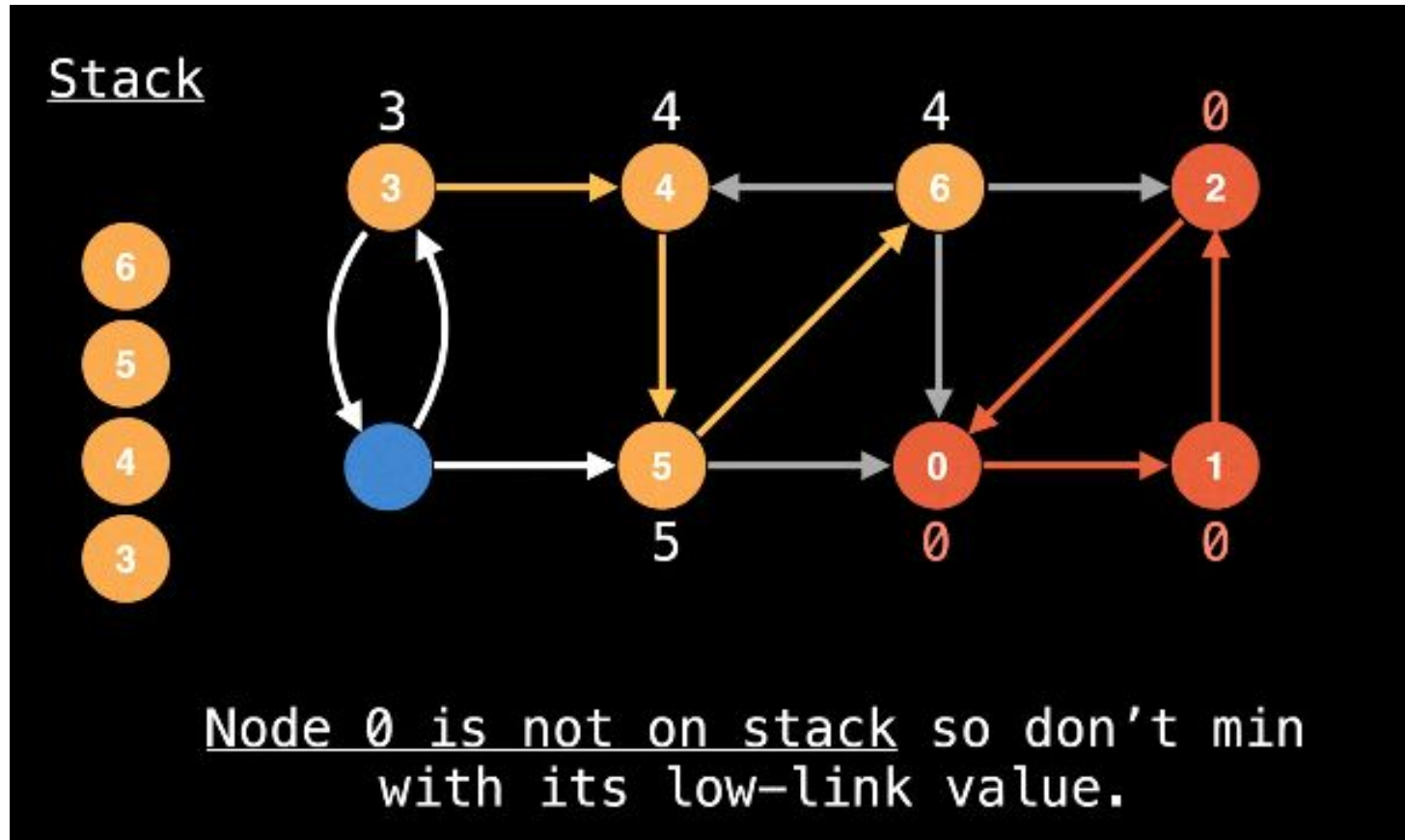
Algoritmo de Tarjan



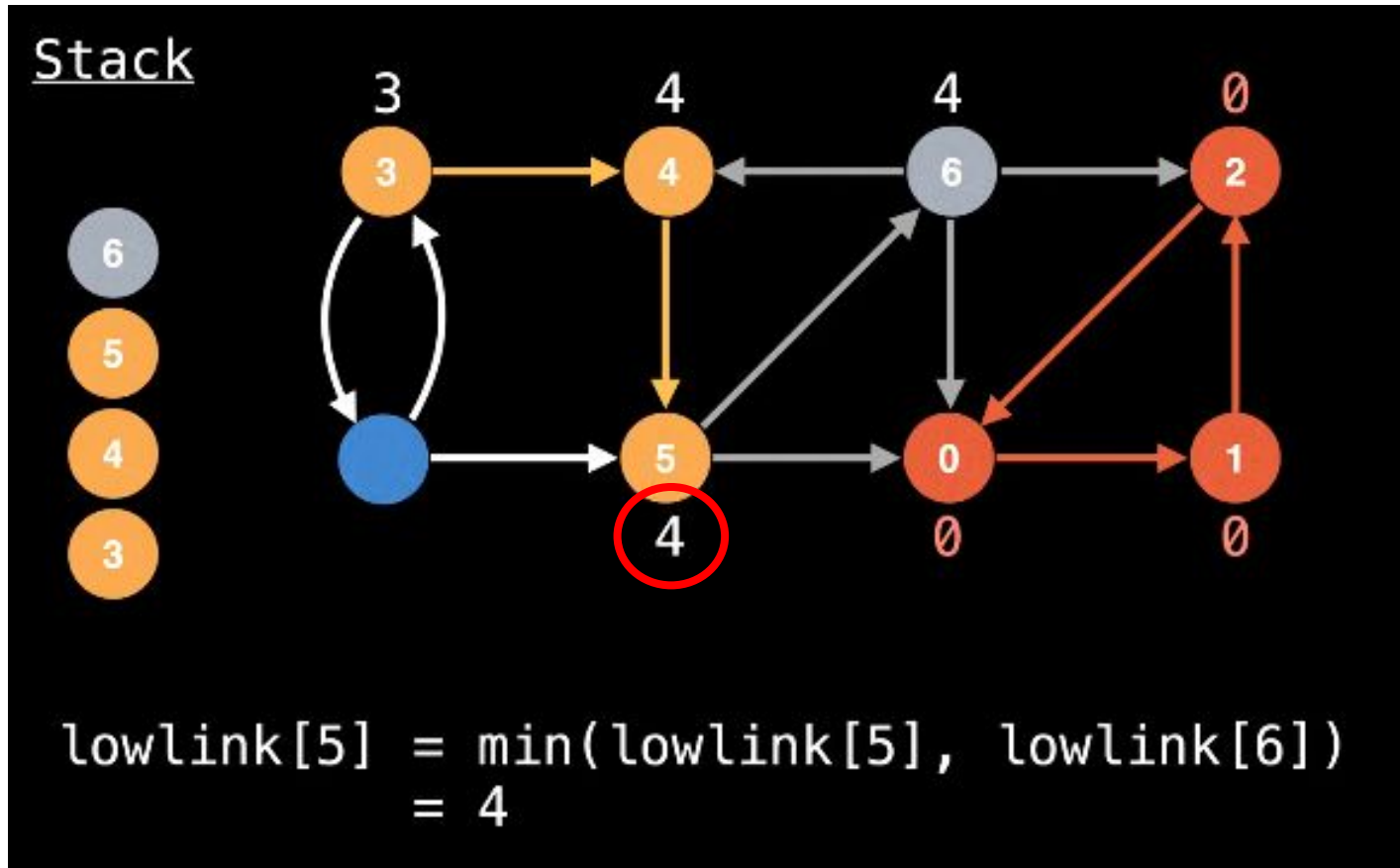
Algoritmo de Tarjan



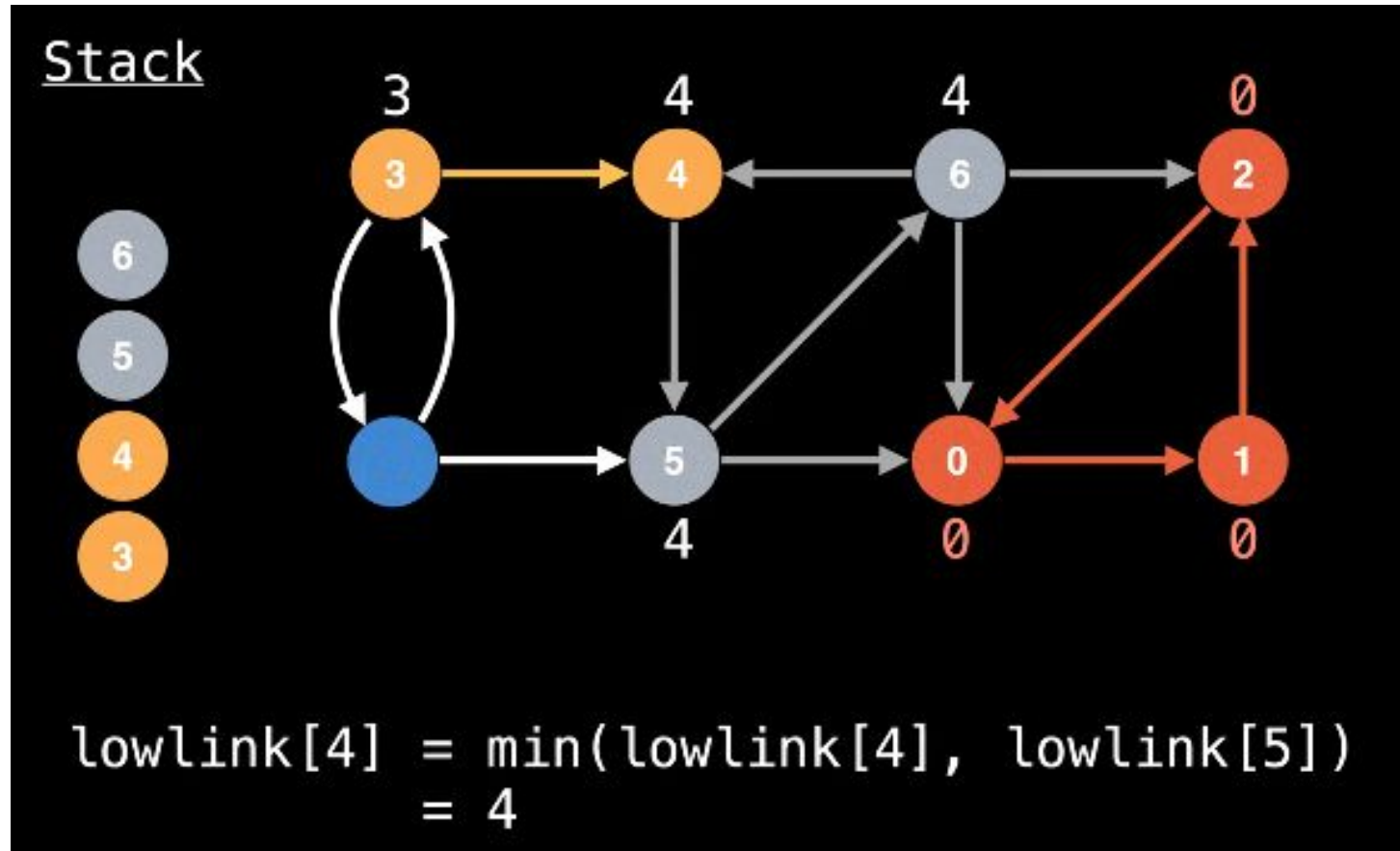
Algoritmo de Tarjan



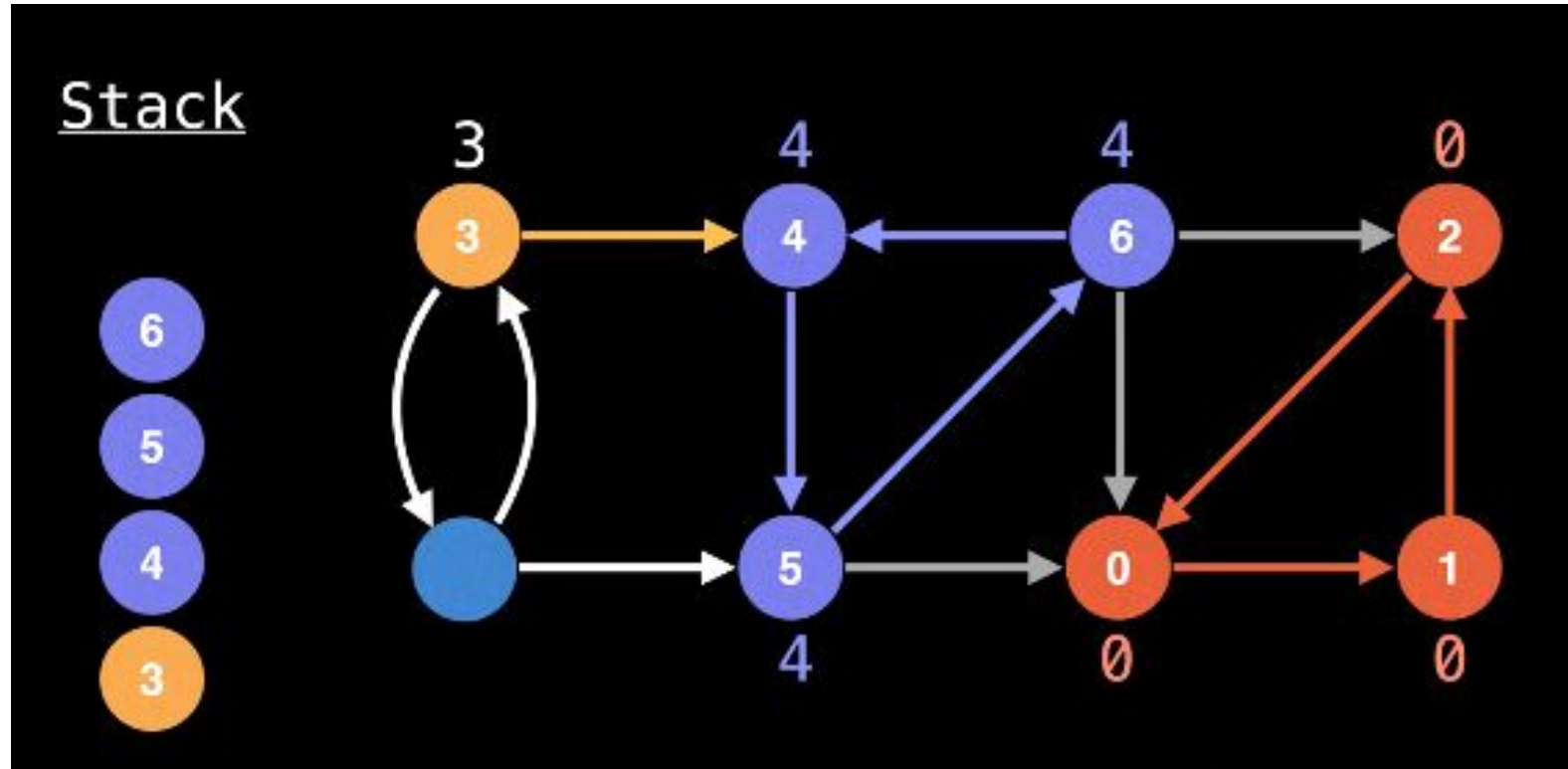
Algoritmo de Tarjan



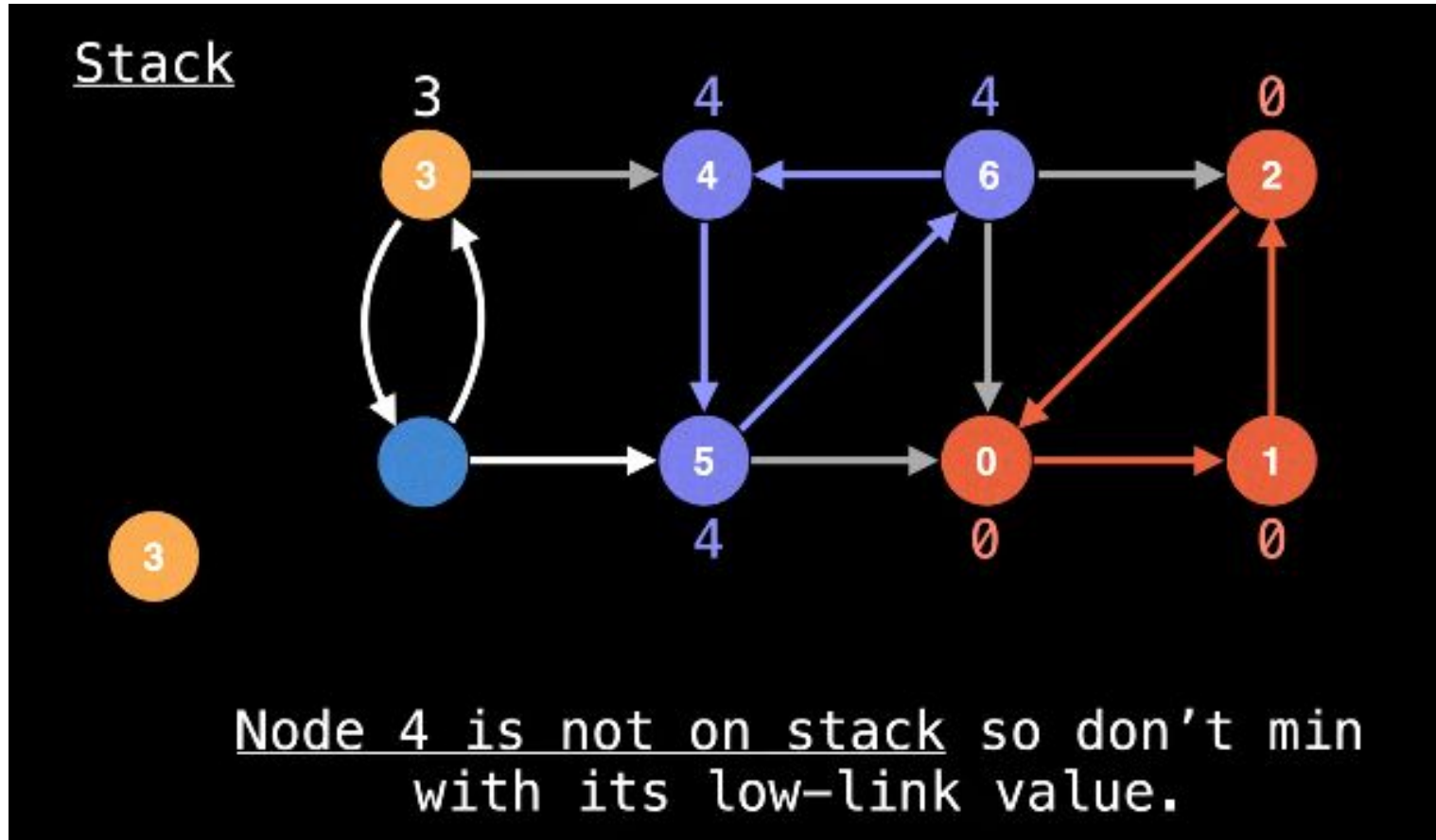
Algoritmo de Tarjan



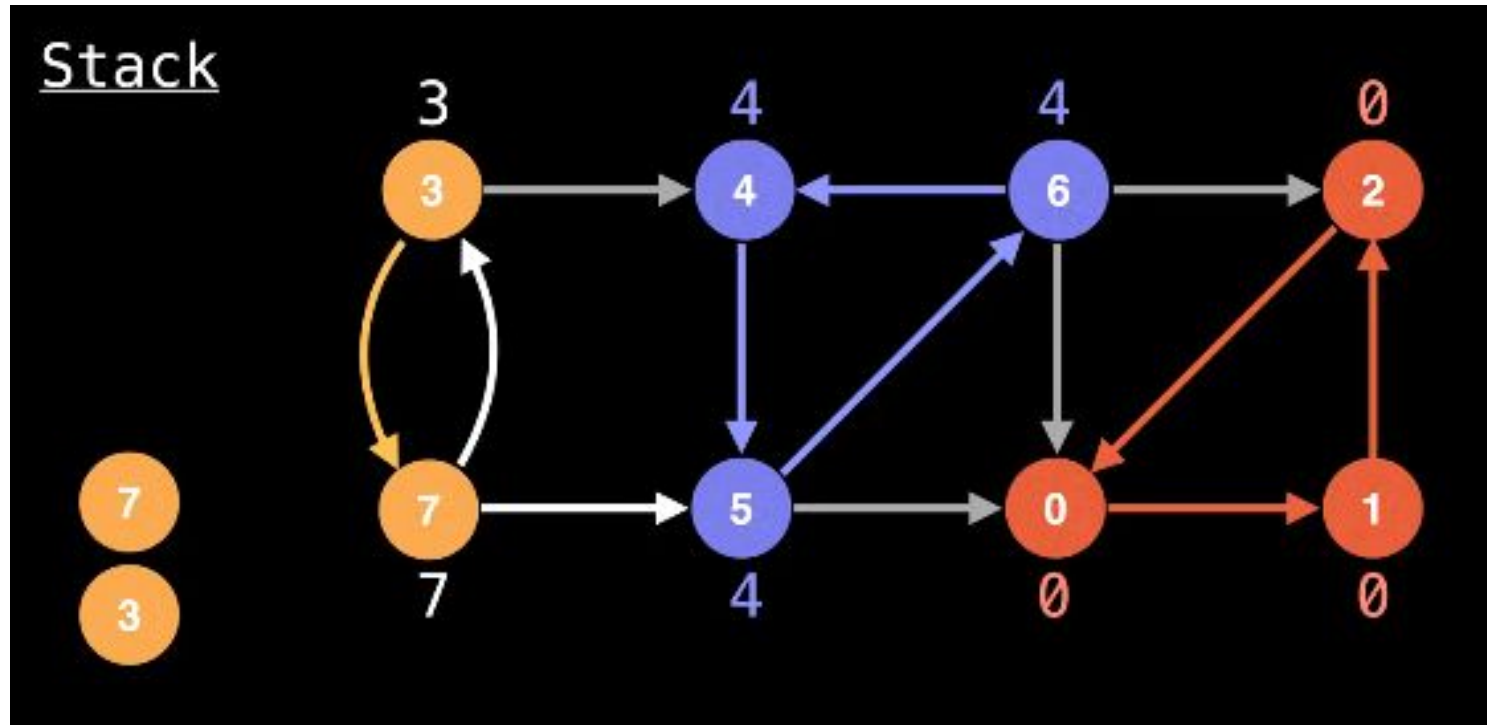
Algoritmo de Tarjan



Algoritmo de Tarjan

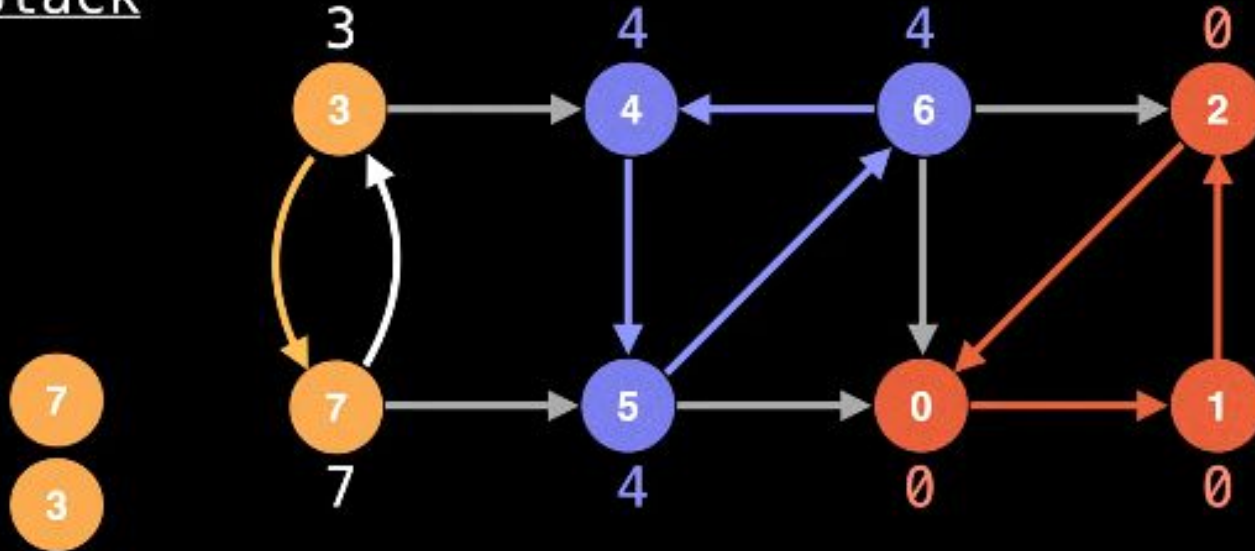


Algoritmo de Tarjan



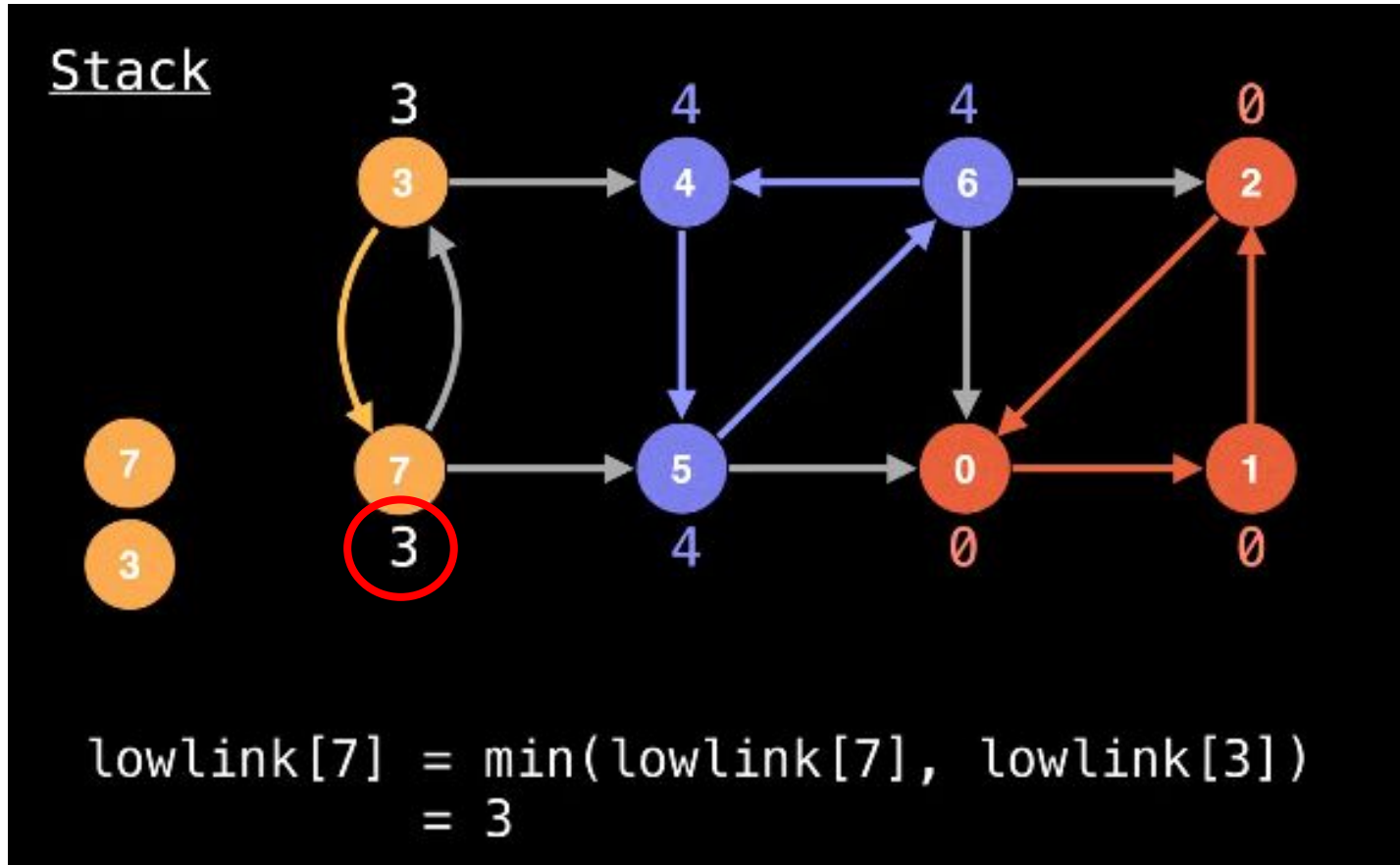
Algoritmo de Tarjan

Stack

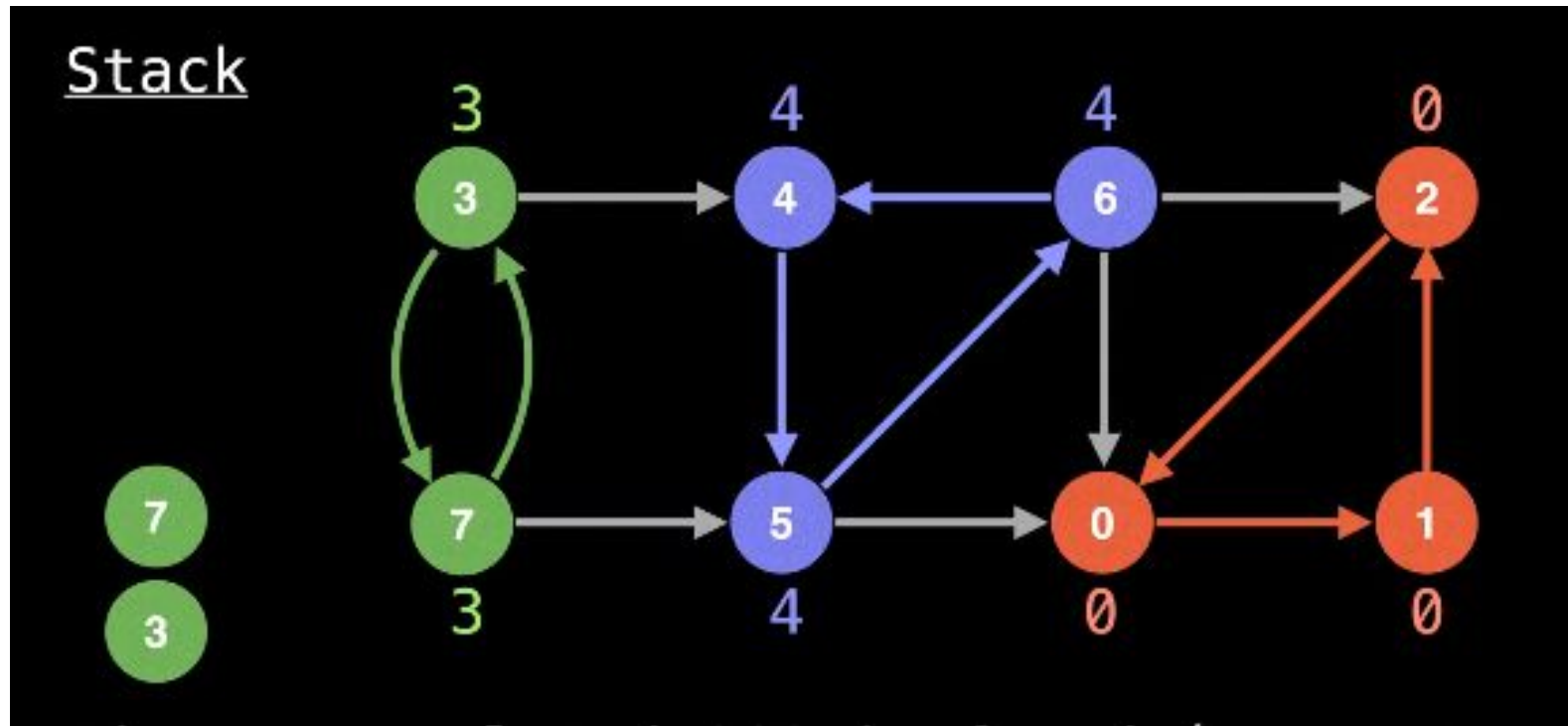


Node 5 is not on stack so don't min
with its low-link value.

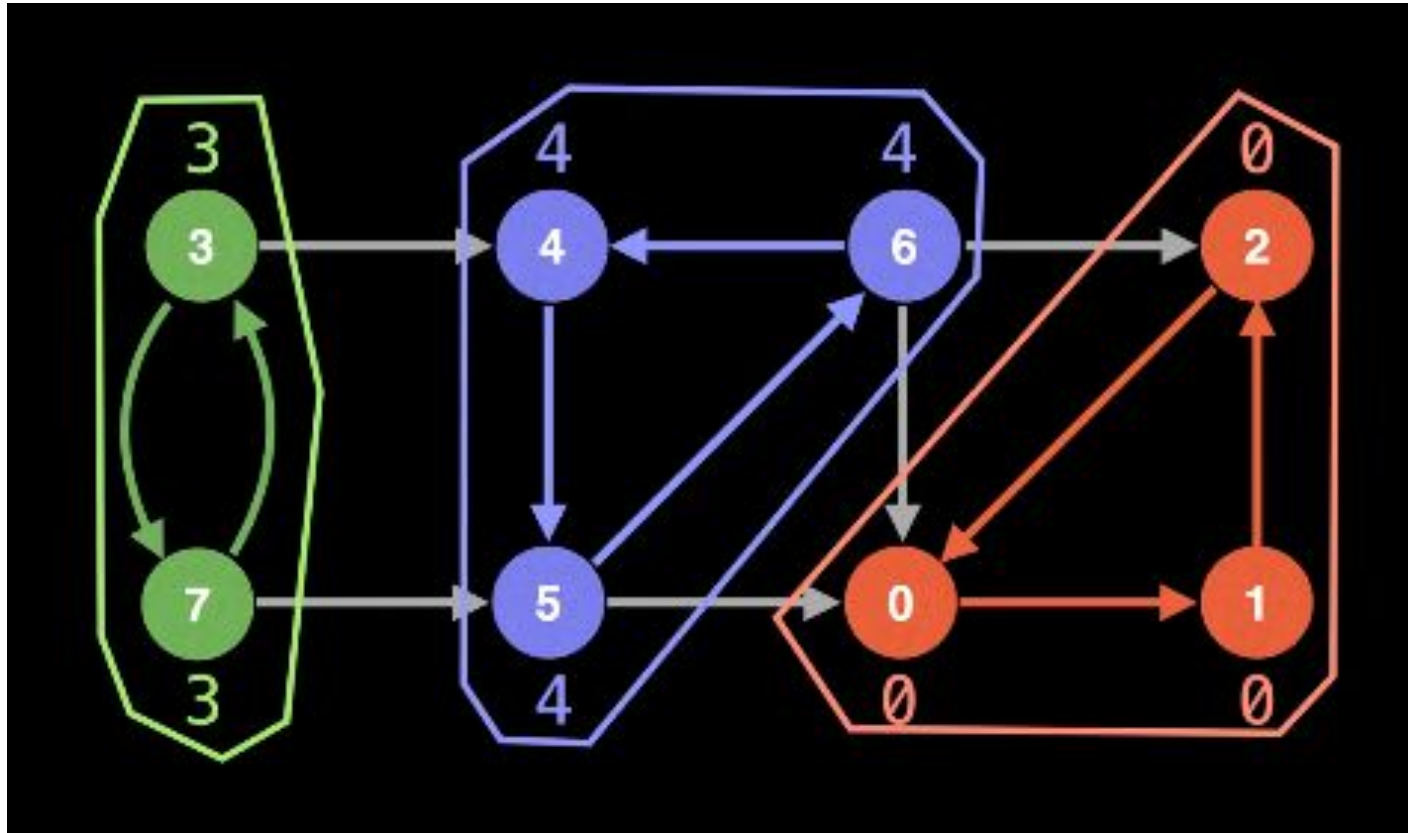
Algoritmo de Tarjan



Algoritmo de Tarjan



Algoritmo de Tarjan



Algoritmo de Tarjan

Código: <https://pastebin.com/zZ1LJhk5>